Le Transformer

TALN Semaine 7

Merci à Dan Jurafsky pour la plupart des diapos de cette semaine !

Nous étudierons les modèles de langage, ajoutant à leur complexité à chaque niveau :

MODÈLES DE LANGUE

- **Modèles de langage N-gram**—Modèles probabilistes de langage contextualisé
- **Embeddings** — Sémantiques vectorielles
- **Modèles de langage neuronaux récurrents, LSTMs**—Modèles de langage neuronaux
- **Transformers** — Grand modèle de langage ou LLM
- **Modèles encodeurs** — Tâches de notation
- **Modèles encodeur-décodeurs** — tâches génératives
- **Peaufinage et apprentissage en-contexte**— le paradigme actuel de modèle fondateur

Plan pour aujourd'hui

1. Révision de la multiplication de matrice
2. Attention simplifiée
3. Attention multi-têtes (multi-head)
4. Bloc Transformer et flux résiduel
5. Entrées de Tranformer et les embeddings positionels
6. Les sorties de Transformer et la tête de modélisation de language
7. *Exercices de groupe*

Notation dans le livre

Récapitulatif rapide de notre notation et de notre multiplication matrice-matrice et matrice-vecteur

- Que $\mathbf{W}$ dénote une matrice $d \times p$
- Que $\mathbf{X}$ dénote une matrice $n \times d$
- Que $\mathbf{x}$ dénote un vecteur $1 \times d$ (rangé)
- $\mathbf{x}^\top$ est un vecteur $d \times 1$ (colonne)
- Notez que: $(\mathbf{XW})^\top = \mathbf{W}^\top \mathbf{X}^\top$

Nous devons comprendre :

- $\mathbf{y} = \mathbf{xW}$
- $\mathbf{y}^\top = \mathbf{W}^\top \mathbf{x}^\top$
- $\mathbf{Y} = \mathbf{XW}$
- $\mathbf{Y}^\top = \mathbf{W}^\top \mathbf{X}^\top$

Le papier qui a tout déclenché

Transformateur : un type spécifique d'architecture de réseau, comme un réseau feedforward plus sophistiqué, mais basé sur l'attention

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

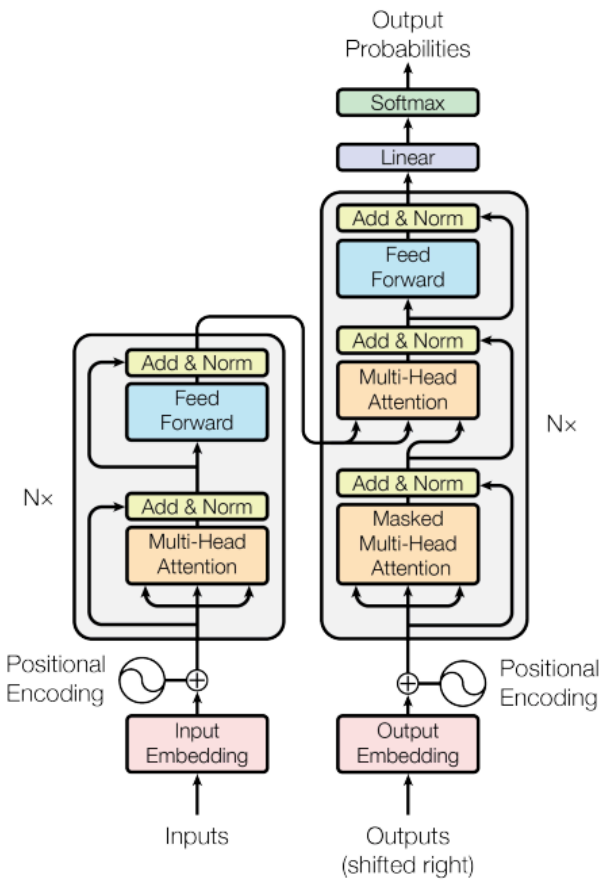
Llion Jones* Google Research llion@google.com

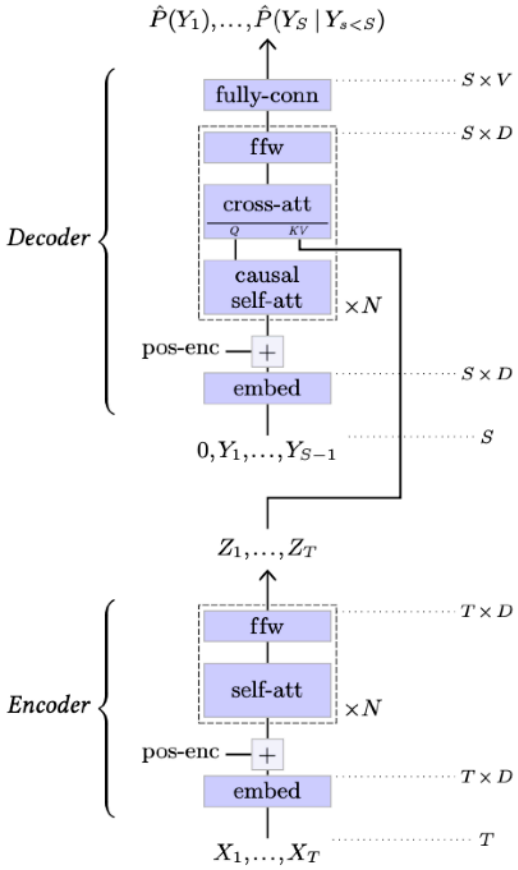
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

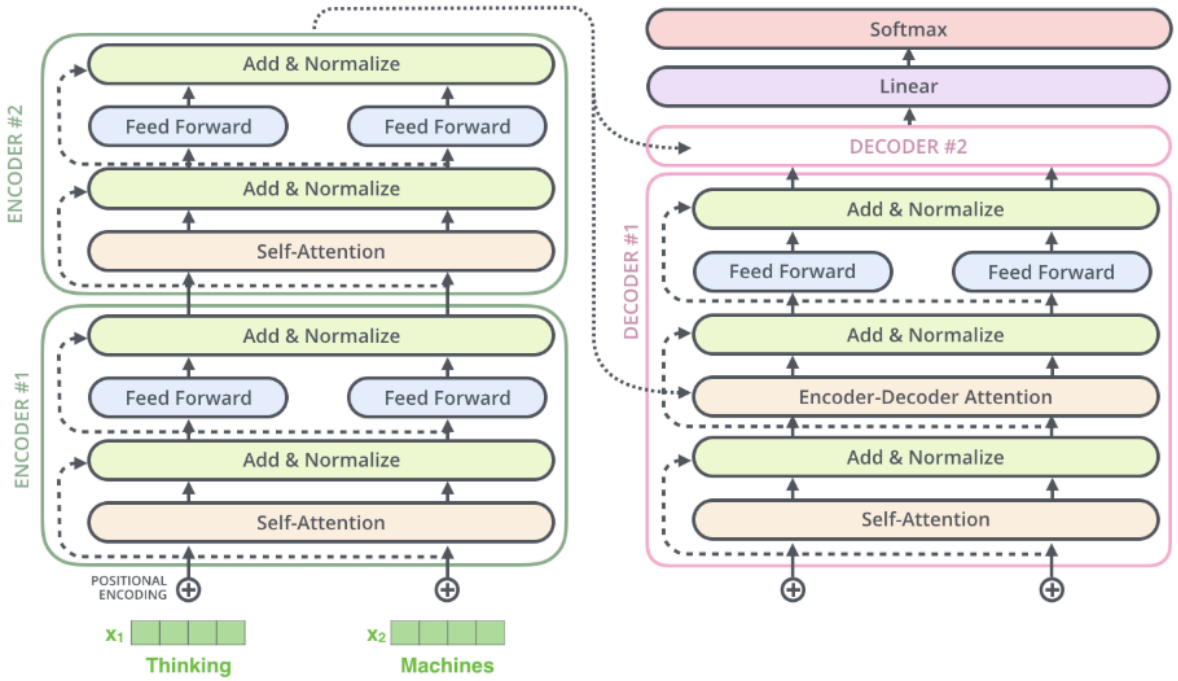
Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Qu’est ce qu’un Transformer?





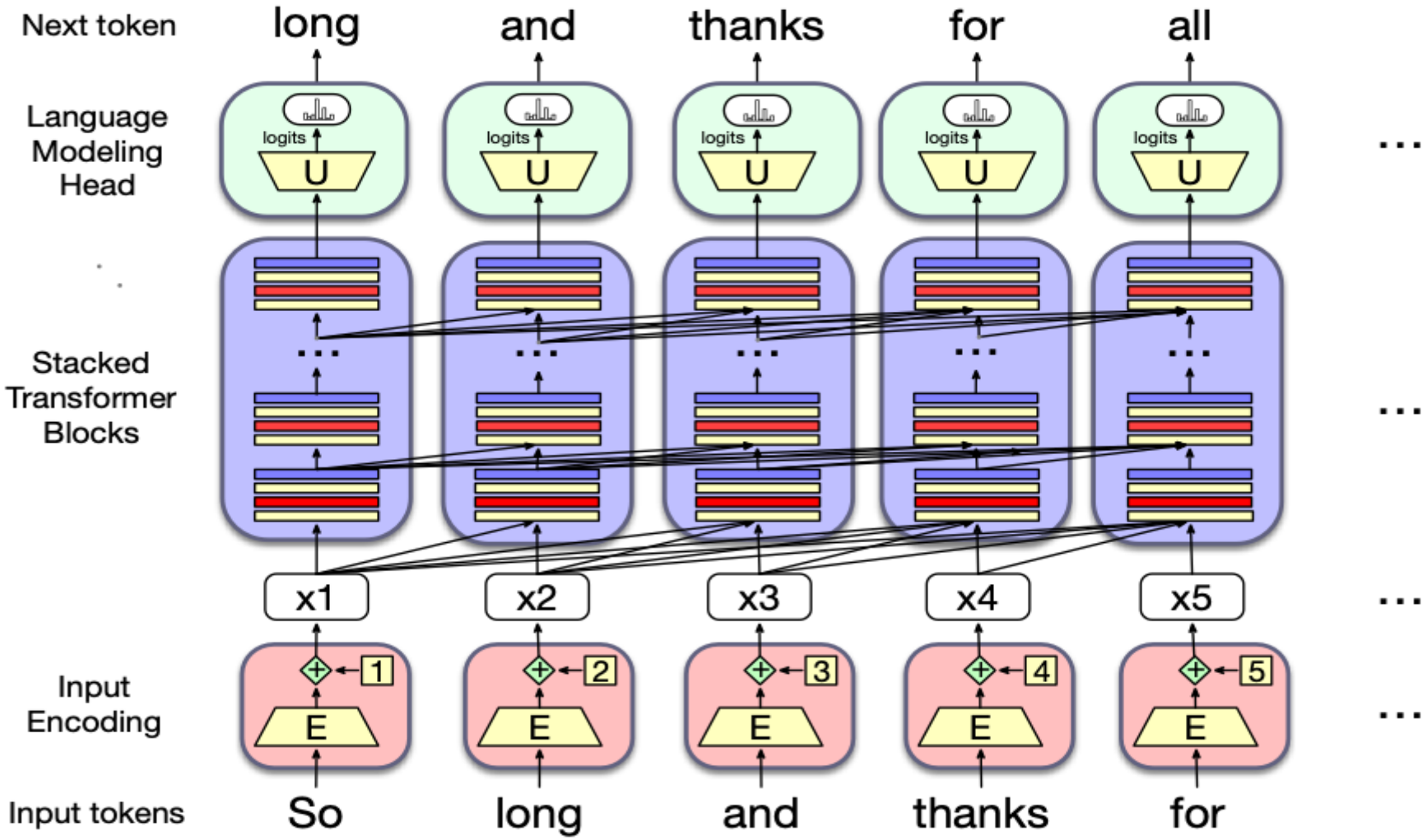


Vaswani et al. 2017 - Attention is all you need

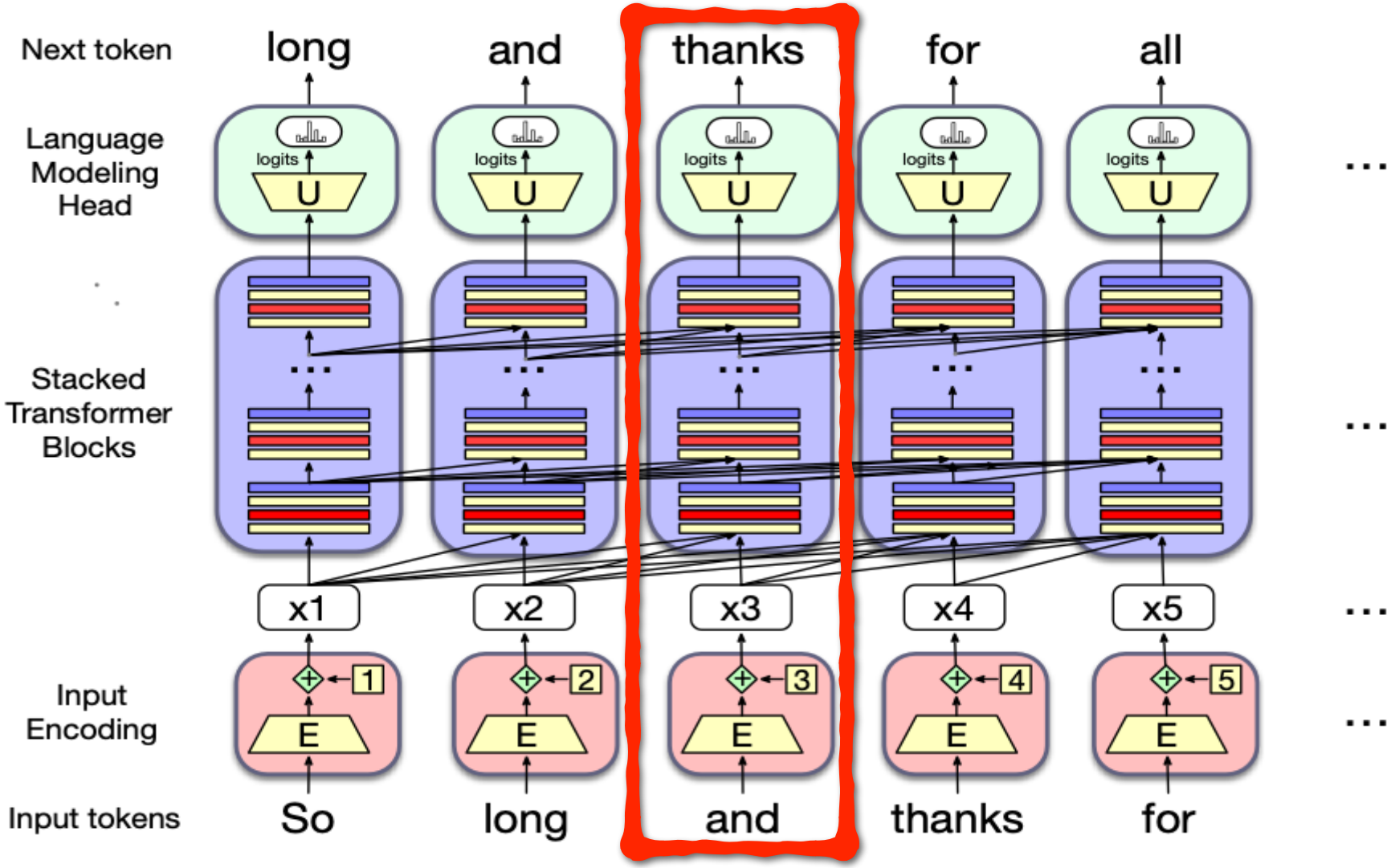
Fleuret et al. 2024 - The little book of DL

Alammar et al. 2018 - The Illustrated Transformer

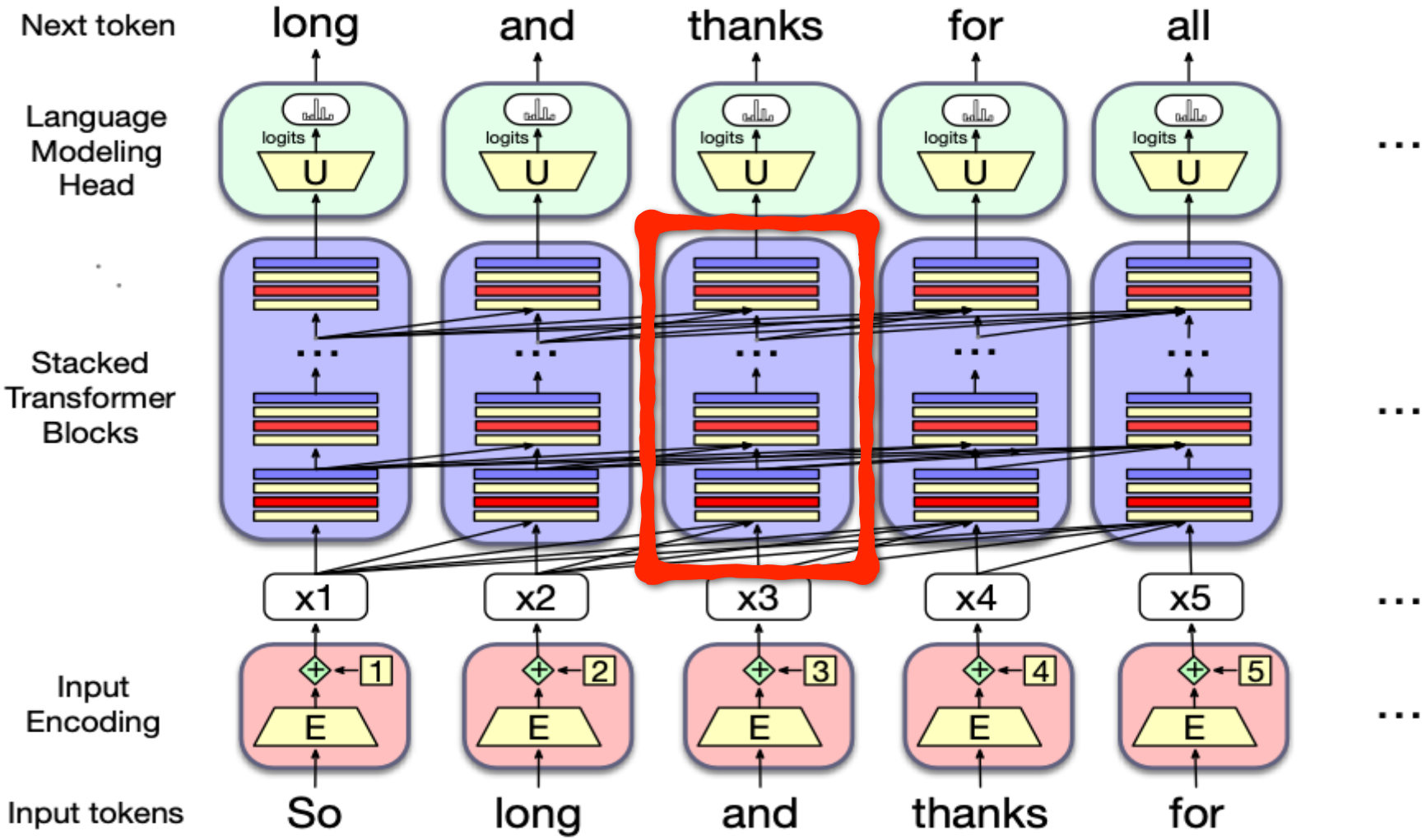
Nous nous en tiendrons à l'illustration suivante



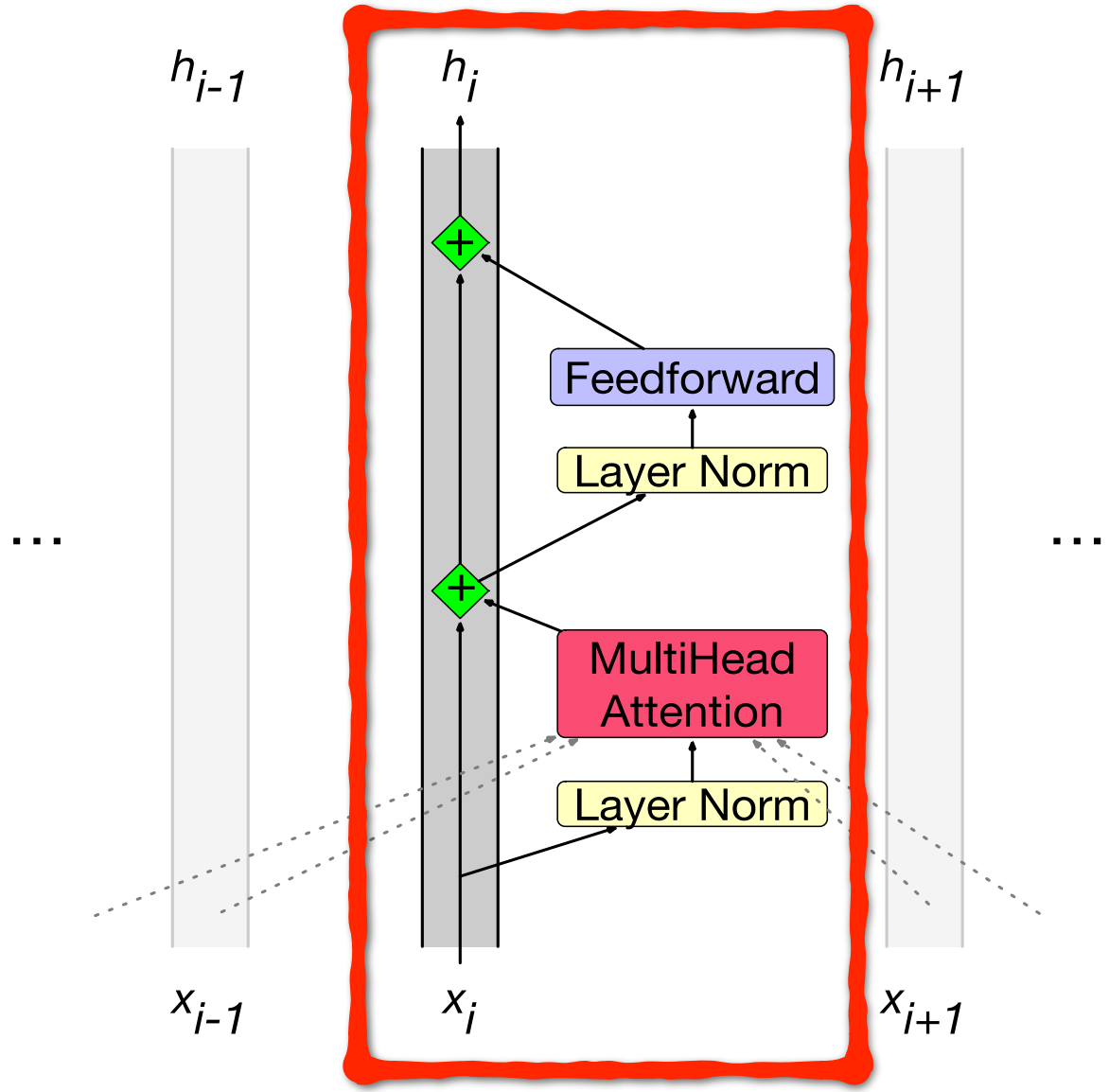
Considérons ce qu'il arrive au embedding d'un mot



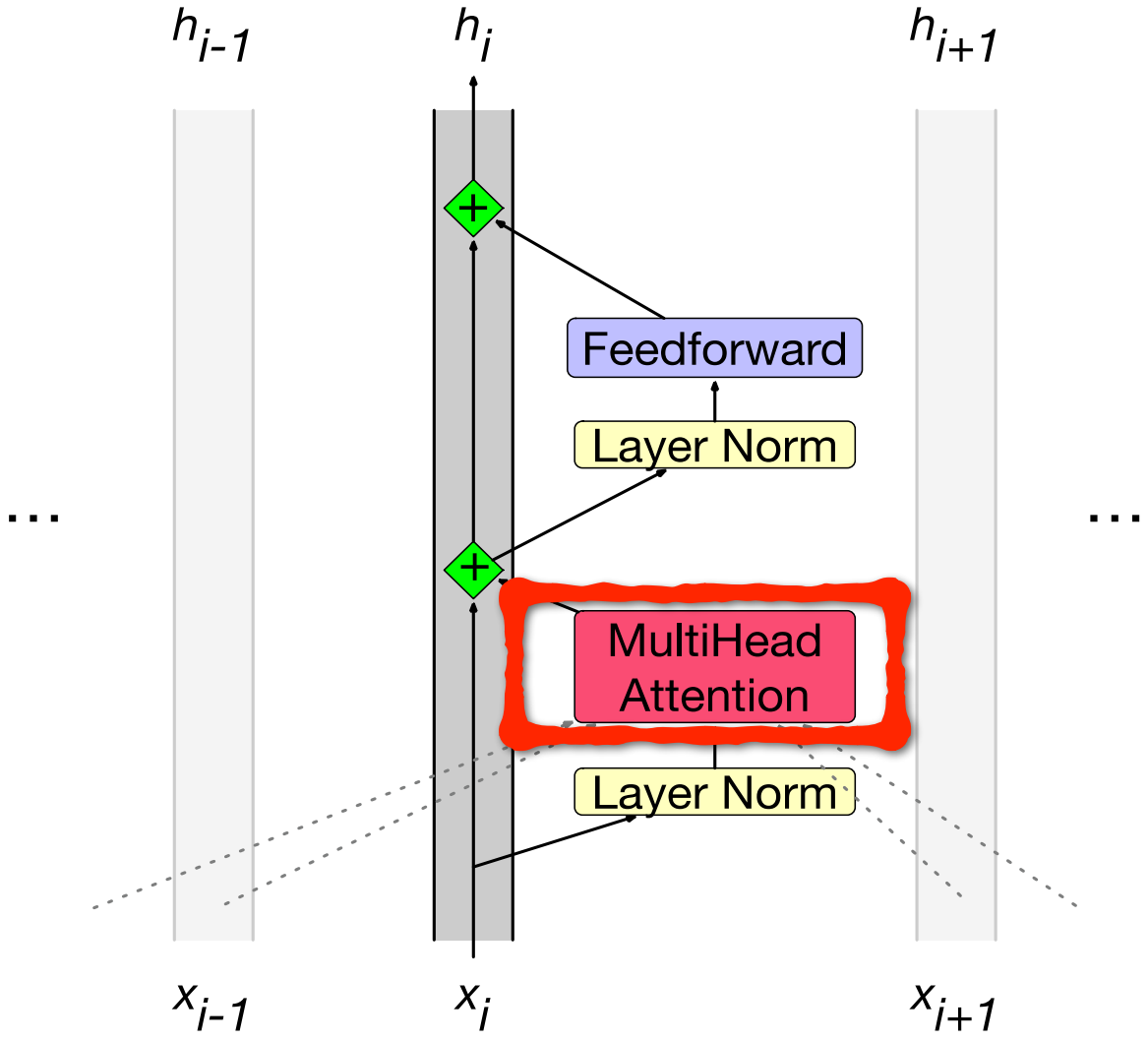
Blocs Transformer



Zoom in : un block transformer



Zoom in : un block transformer - tête d'attention



Intuition de l'attention

- Construire la représentation d'un mot en intégrant sélectivement les informations de tous les mots voisins

- Nous disons qu'un mot “attends to” certains mots voisins plus que d'autres

Problème avec les embeddings statique (word2vec)

Ils sont statiques ! L'embedding d'un mot ne reflète pas la façon dont son sense change dans le contexte.

```
The chicken didn't cross the road because it was too tired
```

Quelle est le sense représentée dans l’embedding statique de “it" ?

Embeddings contextuels

- Intuition: La représentation du sense d'un mot devrait être différente dans différents contextes !
- **Embedding contextuel**: Chaque mot a un vecteur différent qui exprime un sense différent en fonction des mots environnants
- Comment calculer les embeddings contextuels?
  - **Attention**

Embeddings contextuels

`The chicken didn't cross the road because it`

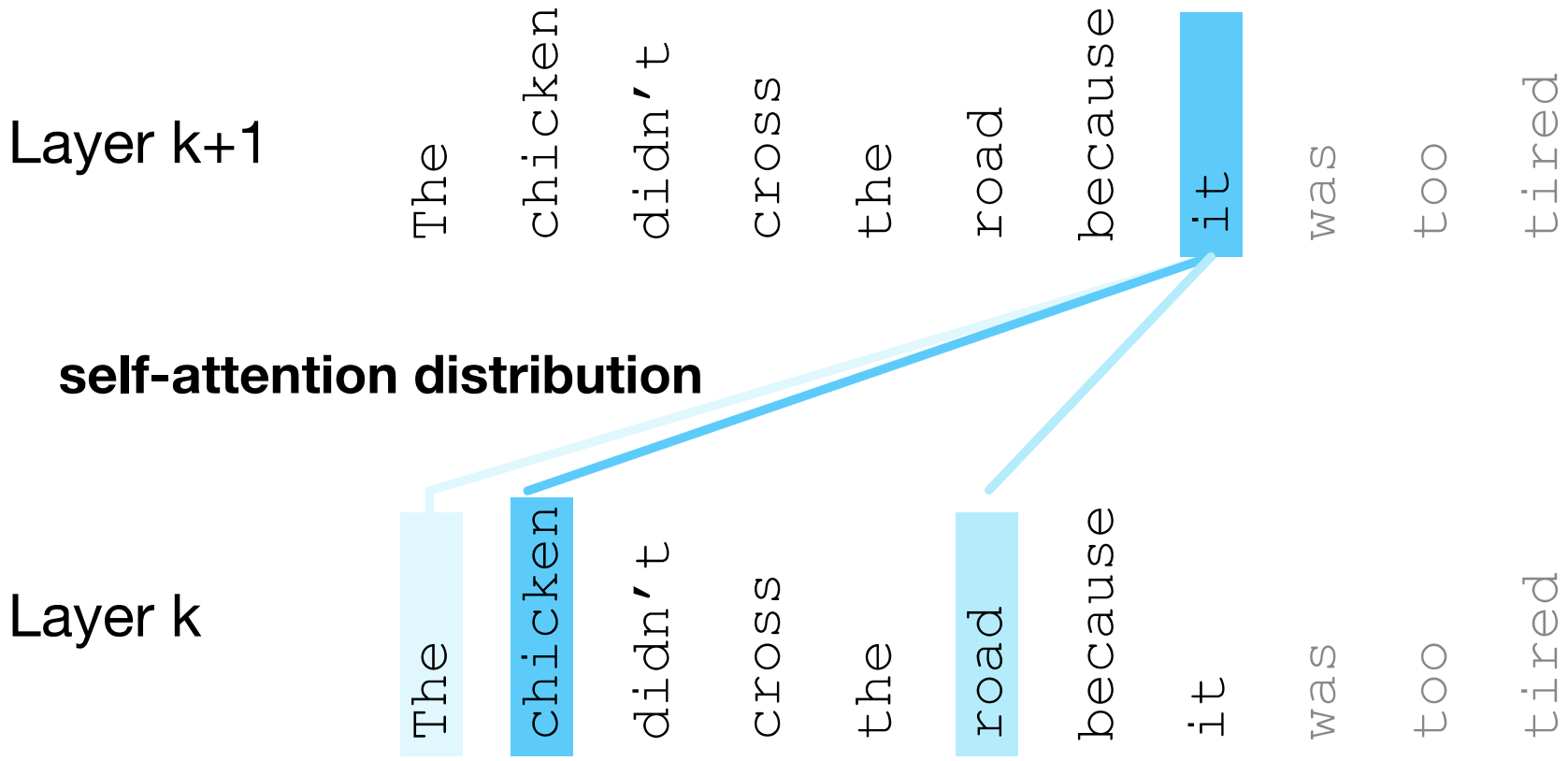
Quelles devraient être les propriétés de "it"?

`The chicken didn't cross the road because it was too` **`tired`**
`The chicken didn't cross the road because it was too` **`wide`**

À ce stade de la phrase, il fait probablement référence à “chicken” ou à “road”

Intuition de l'attention:

columns corresponding to input tokens



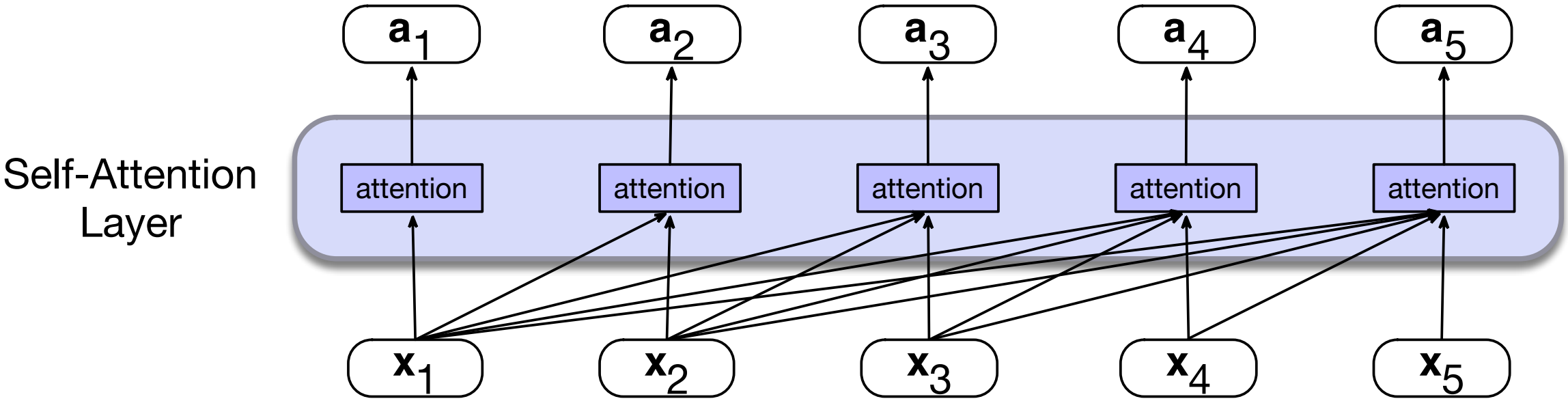
Définition de l'attention

Un mécanisme pour aider à calculer l’embedding d’un token en traitant et en intégrant sélectivement les informations des tokens environnants (à la couche précédente).

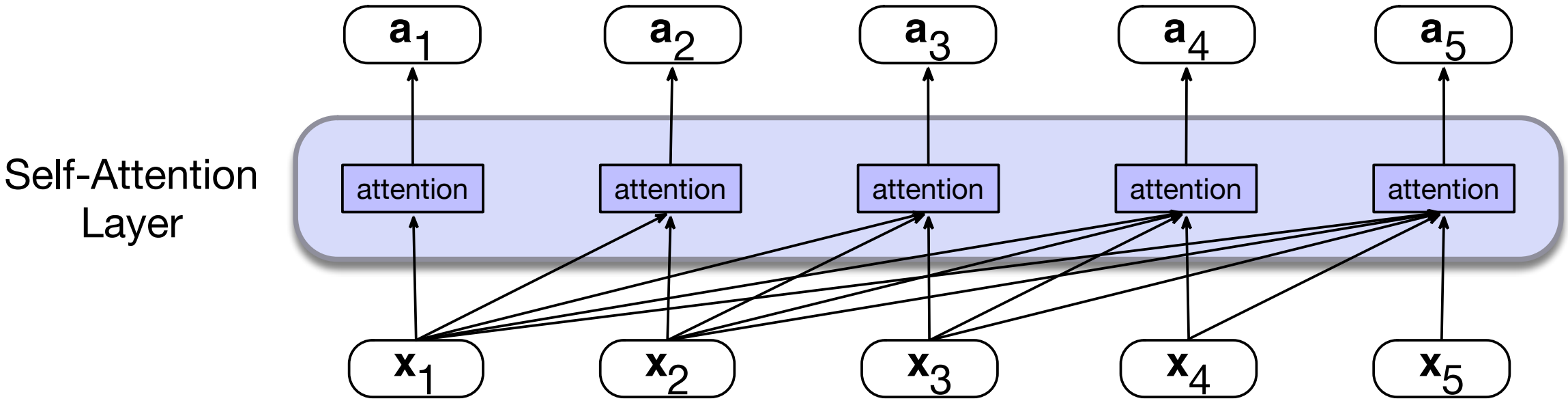
Plus formellement : une méthode pour faire une somme pondérée de vecteurs.

$$\mathbf{a}^i = \sum_{j \leq i} \alpha_{ij} \cdot \mathbf{x}_j$$

L'attention de gauche à droite



Attention can respect time (causal)



$$\mathbf{a}_j = \sum_{i=1}^{n} (\alpha_i \cdot \mathbb{M}(i,j)) \cdot \mathbf{x}_i \qquad \mathbb{M} = \begin{cases} 1 & \text{if} \quad i \leq j \\ 0 & \text{else} \end{cases}$$

Version simplifiée de l'attention : une somme des mots précédents pondérée par leur similitude avec le mot actuel

Étant donné une séquence de token embeddings:

$$\mathbf{x}_1 \quad \mathbf{x}_2 \quad \mathbf{x}_3 \quad \mathbf{x}_4 \quad \mathbf{x}_5 \quad \mathbf{x}_6 \quad \mathbf{x}_7 \quad \mathbf{x}_i$$

Produit: $\mathbf{a}_i$ = somme pondérée de $\mathbf{x}_1$ à $\mathbf{x}_7$ (et $\mathbf{x}_i$), pondérée par leur score de similitude avec $\mathbf{x}_i$

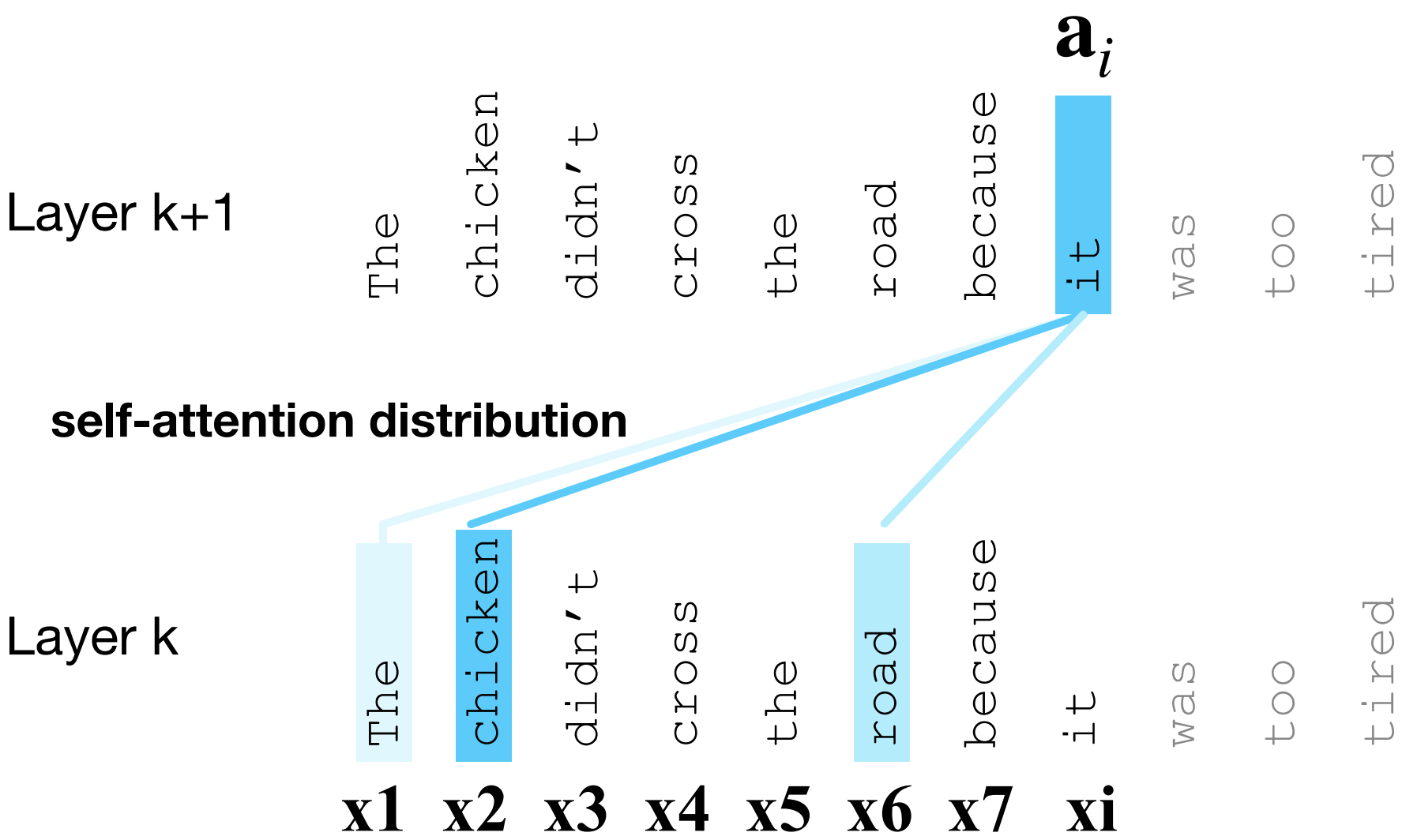
$$\mathbf{a}_i = \sum_{j \leq i} \alpha_j \cdot \mathbf{x}_j$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\alpha = \text{softmax}([\text{score}(\mathbf{x}_i, \mathbf{x}_j) \text{ for j in } 1 \ldots 7, i])$$

Intuition de l'attention:

columns corresponding to input tokens

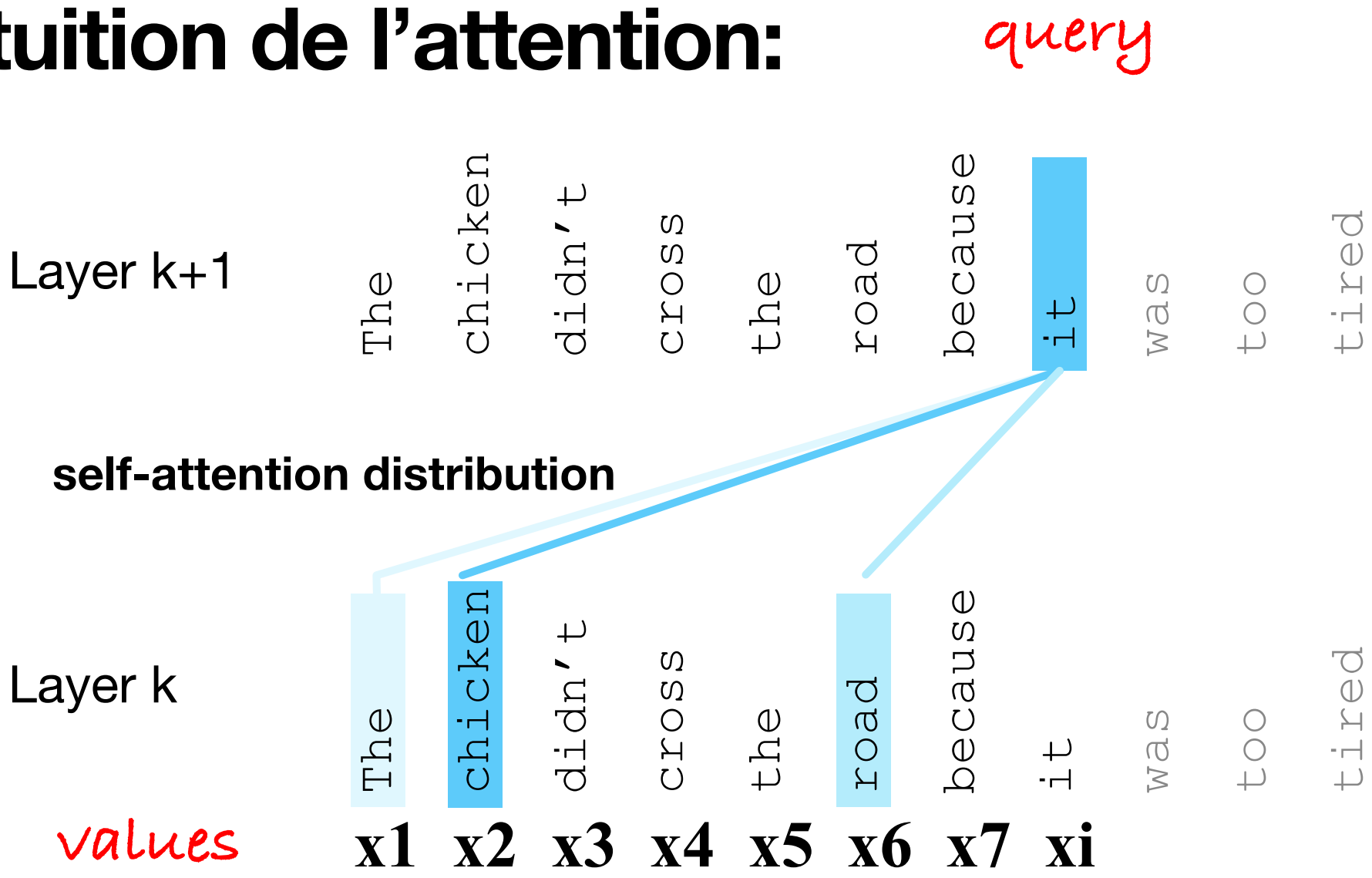


Une tête d'attention réelle est un peu plus compliquée

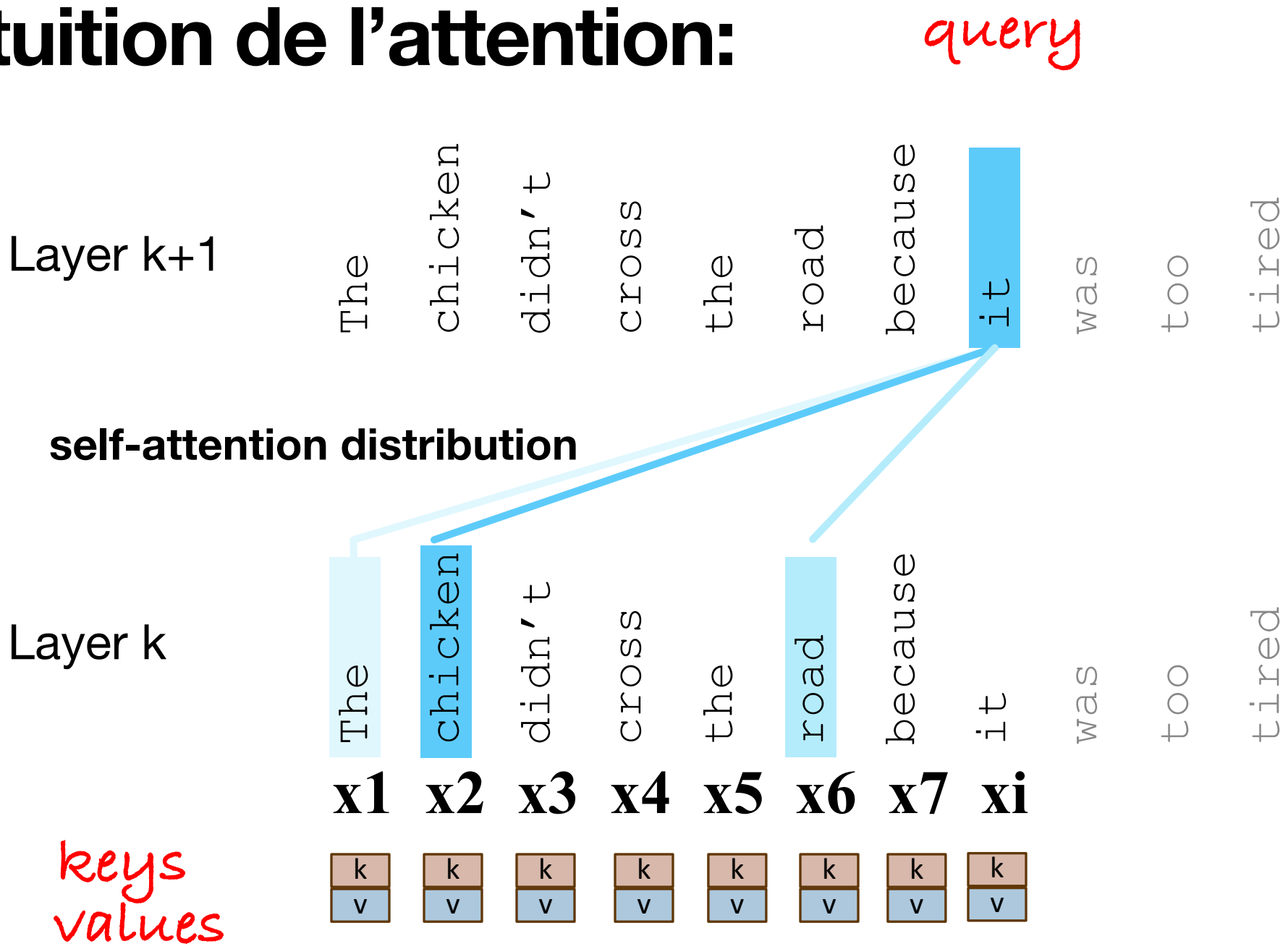
Idée: au lieu d'utiliser des vecteurs (comme x_i et x_4) directement, Nous représenterons 3 rôles distincts que chaque vecteur $\mathbf{x}_i$ joue:

- **query**: Comme l'*élément actuel* étant comparé aux entrées précédentes.
- **key**: Comme un *élément précédent* qui est comparée à l'élément actuel pour déterminer une similitude
- **value**: *Une valeur* d'un élément précédent qui rentre dans la *somme pondérée*

Intuition de l'attention:



Intuition de l'attention:



Tête D'Attention Réelle

Nous utiliserons des matrices pour projeter chaque vecteur $\mathbf{x}_i$ dans une représentation de son rôle en tant que query, key, value:

- **query**: $\mathbf{W}^Q$
- **key**: $\mathbf{W}^K$
- **value**: $\mathbf{W}^V$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \qquad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \qquad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

Tête D'Attention Réelle

Les 3 représentations de $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \qquad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \qquad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

Pour calculer la similitude de l'élément actuel $\mathbf{x}_i$ avec un élément antérieur $\mathbf{x}_j$, nous utiliserons un produit scalaire entre $\mathbf{q}_i$ et $\mathbf{k}_j$.

Et au lieu d'additionner $\mathbf{x}_j$, nous faisons la somme des $\mathbf{v}_j$

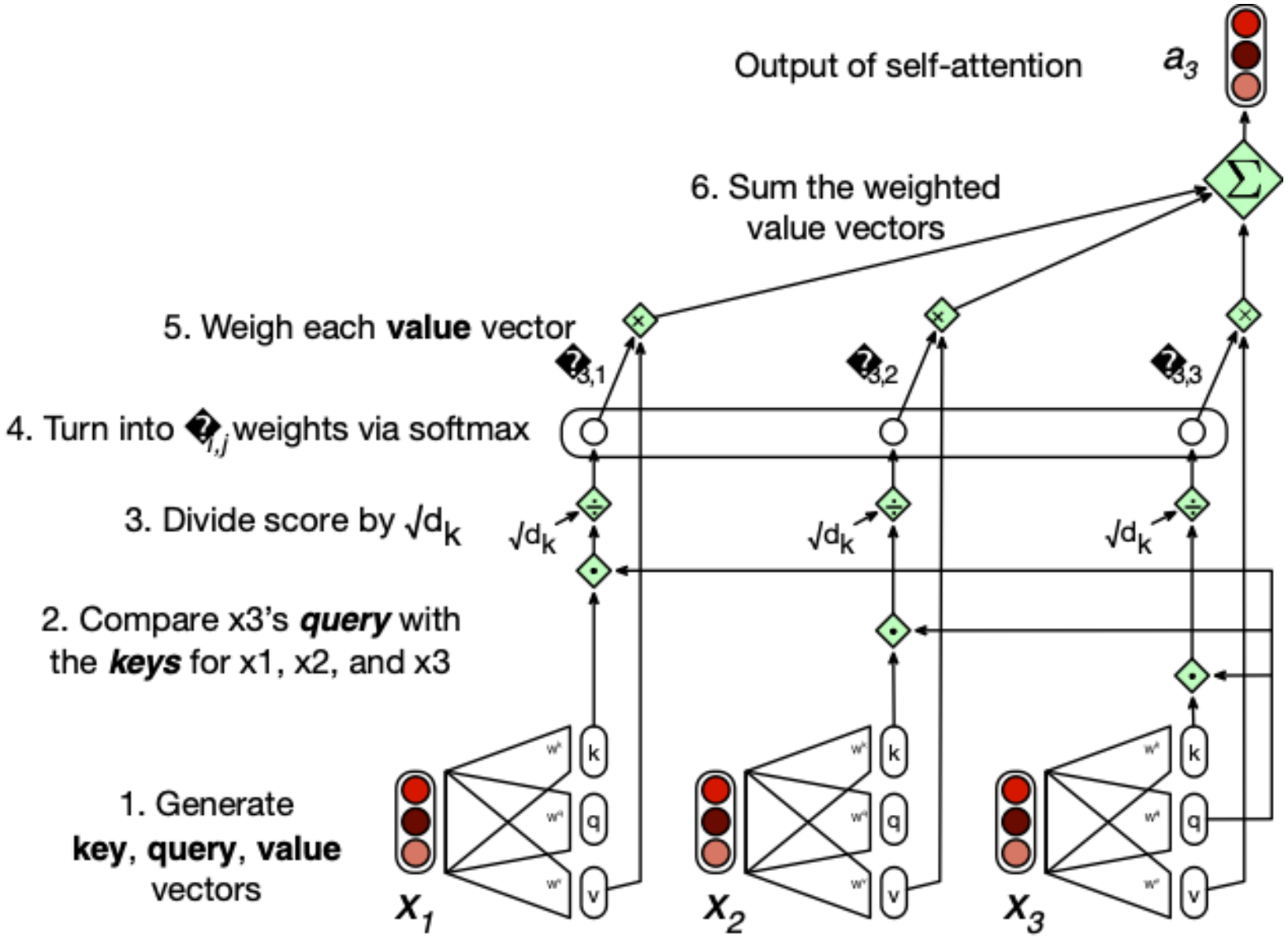
Équations finales pour une tête d'attention

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \qquad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \qquad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \mathbf{k}_j^\top}{\sqrt{d^k}} \qquad \alpha = \text{softmax}([\text{score}(\mathbf{x}_i, \mathbf{x}_j) \forall\ j \leq i])$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_j \cdot \mathbf{v}_j$$

Exemple : calcul de la valeur de a3



Attention multi-têtes

- Au lieu d'une tête d'attention, nous en aurons beaucoup !
- Intuition : chaque tête peut considérer le contexte à des fins différentes
  - Par exemple, différentes relations ou patterns linguistiques dans le contexte

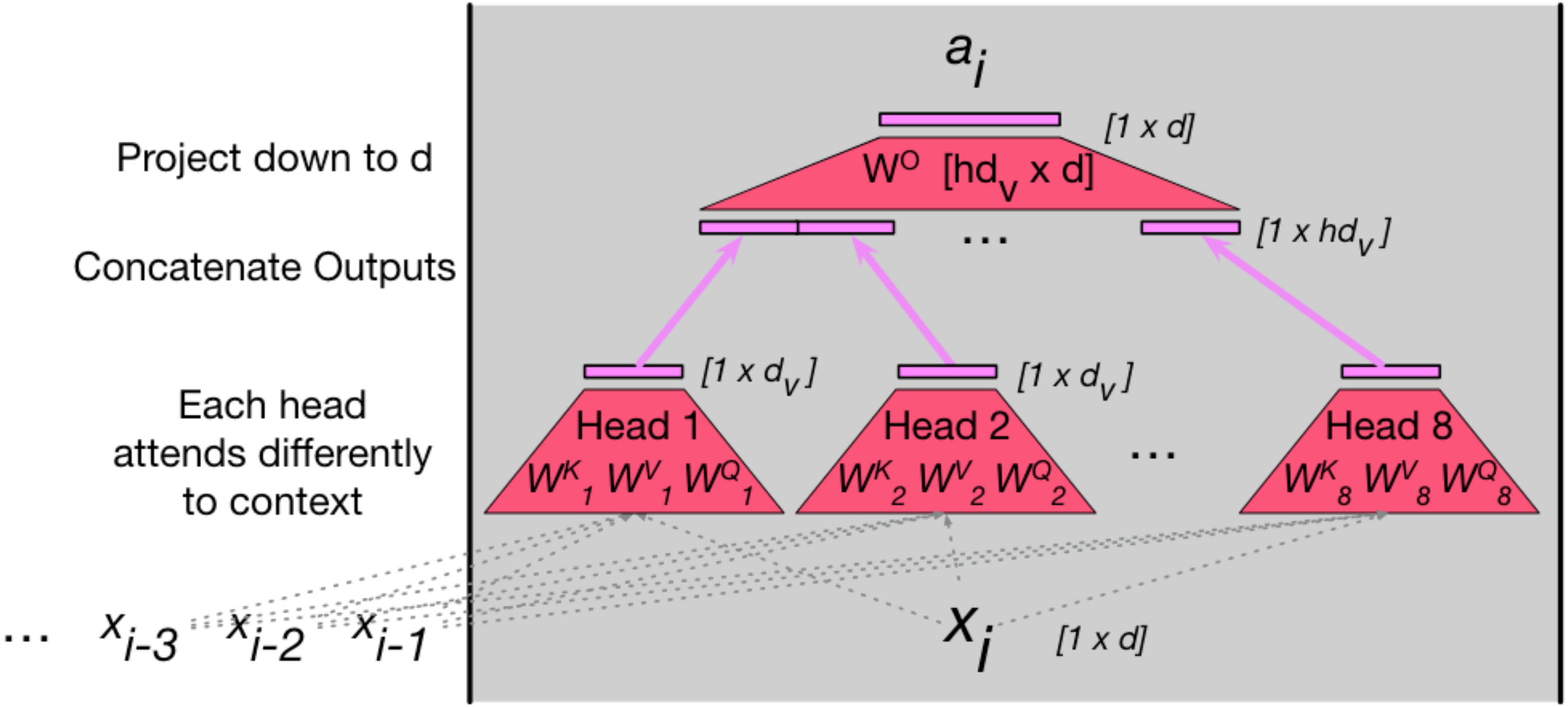
$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{Qc} \quad \mathbf{k}_i^c = \mathbf{x}_i \mathbf{W}^{Kc} \quad \mathbf{v}_i^c = \mathbf{x}_i \mathbf{W}^{Vc}$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \mathbf{k}_j^{c\top}}{\sqrt{d^k}} \qquad \alpha_i^c = \text{softmax}([\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) \forall\ j \leq i])$$

$$\textbf{head}_i^c = \sum_{j \leq i} \alpha_{i,j}^c \cdot \mathbf{v}_j^c \qquad \mathbf{a}_i = (\textbf{head}^1 \oplus \textbf{head}^2 \ldots \oplus \textbf{head}^h)\mathbf{W}^O$$

$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \ldots, \mathbf{x}_n]) = \mathbf{a}_i$$

Attention multi-têtes



Calcul parallèle à l'aide de X

Pour le bloc transformer/d’attention, nous avons calculé un **seul** ouput à un **seul** pas de temps *i* dans un **seul** flux résiduel.

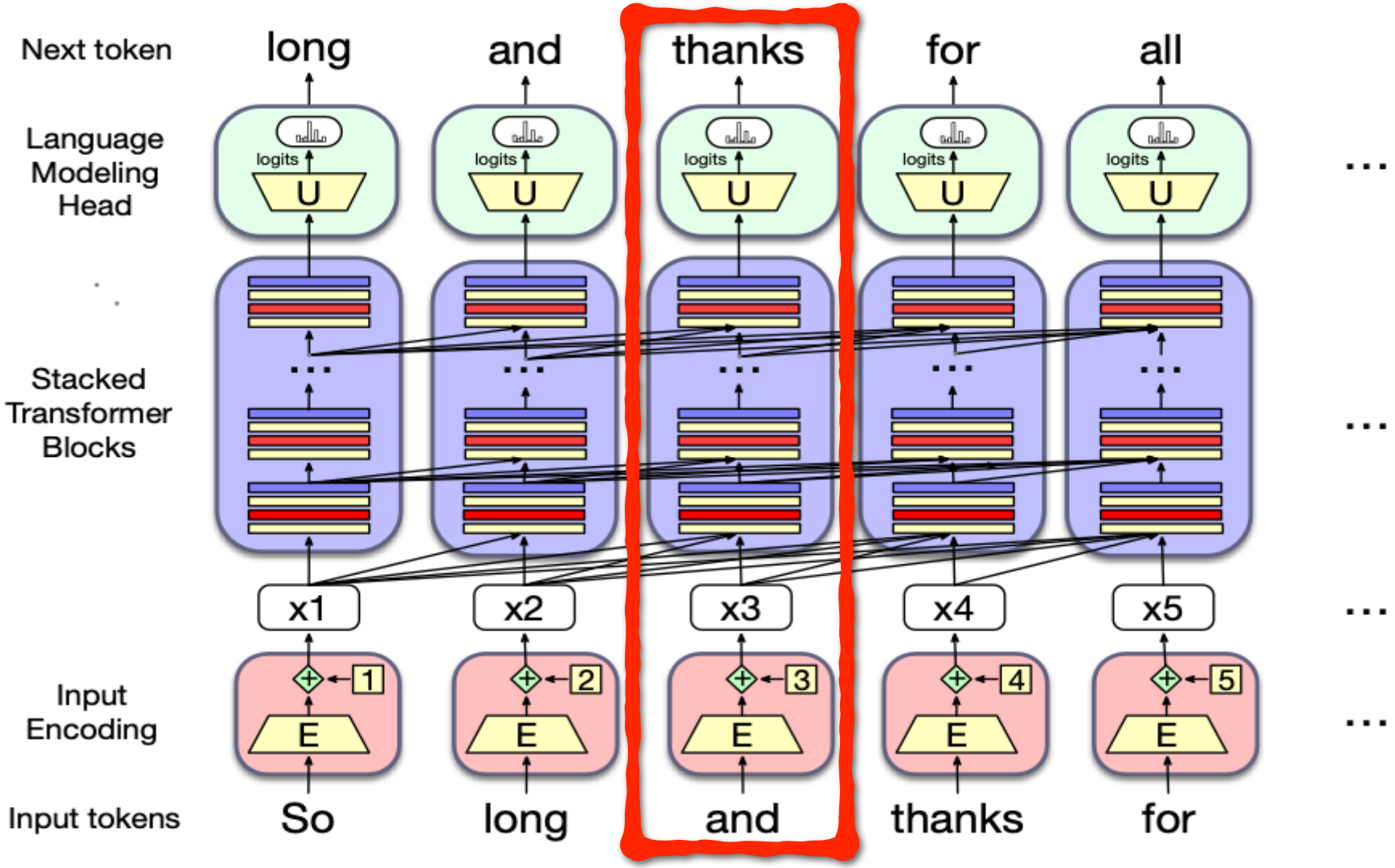
Mais nous pouvons emballer les N tokens de la séquence d'entrée dans une seule matrice X de taille [N × d].

Chaque rangée de X est l’embedding d'un token de l'input.

X peut avoir des lignes de 1K-32K, chacune de la dimensionnalité de l’embedding *d* (la **dimension du modèle**)

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{Q} \qquad \mathbf{K} = \mathbf{X}\mathbf{W}^{K} \qquad \mathbf{V} = \mathbf{X}\mathbf{W}^{V}$$

Considérons ce qu'il arrive au embedding d'un mot



QK^T

Maintenant, vous pouvez faire une seule matrice multiplier pour combiner Q et K^T

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \mathbf{k}_j^\top}{\sqrt{d^k}}$$

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d^k}}$$

N

q1•k1	q1•k2	q1•k3	q1•k4
q2•k1	q2•k2	q2•k3	q2•k4
q3•k1	q3•k2	q3•k3	q3•k4
q4•k1	q4•k2	q4•k3	q4•k4

N

Calcul parallèle de l’attention

- Utilizez des matrices **Q** et **K** pour calculer les scores, prenez le softmax, puis multipliez le résultat par **V**, ce qui donne une matrice de forme $N \times d$
  - Un vecteur d'attention pour chaque token

$$\mathbf{A} = \text{softmax}\left(\mathbb{M}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d^k}}\right)\right)\mathbf{V}$$

Masquer l'avenir

- Quelle est cette fonction de masque ? $\mathbf{QK}^\top$ a un score pour chaque paire de query-key, y compris celles qui suivent le query.
- Deviner le mot suivant est assez simple si vous le connaissez déjà !

$$\mathbf{A} = \text{softmax}\left(\mathbb{M}\left(\frac{\mathbf{QK}^\top}{\sqrt{d^k}}\right)\right)\mathbf{V}$$

Masquer l'avenir

Ajouter $-\infty$ Aux cellules du triangle supérieur

Le log softmax les transformera en 0

$$\mathbf{A} = \text{softmax}\left(\mathbb{M}\left(\frac{\mathbf{QK}^\top}{\sqrt{d^k}}\right)\right)\mathbf{V}$$

N

q1•k1	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4

N

Another point: Attention is quadratic in length

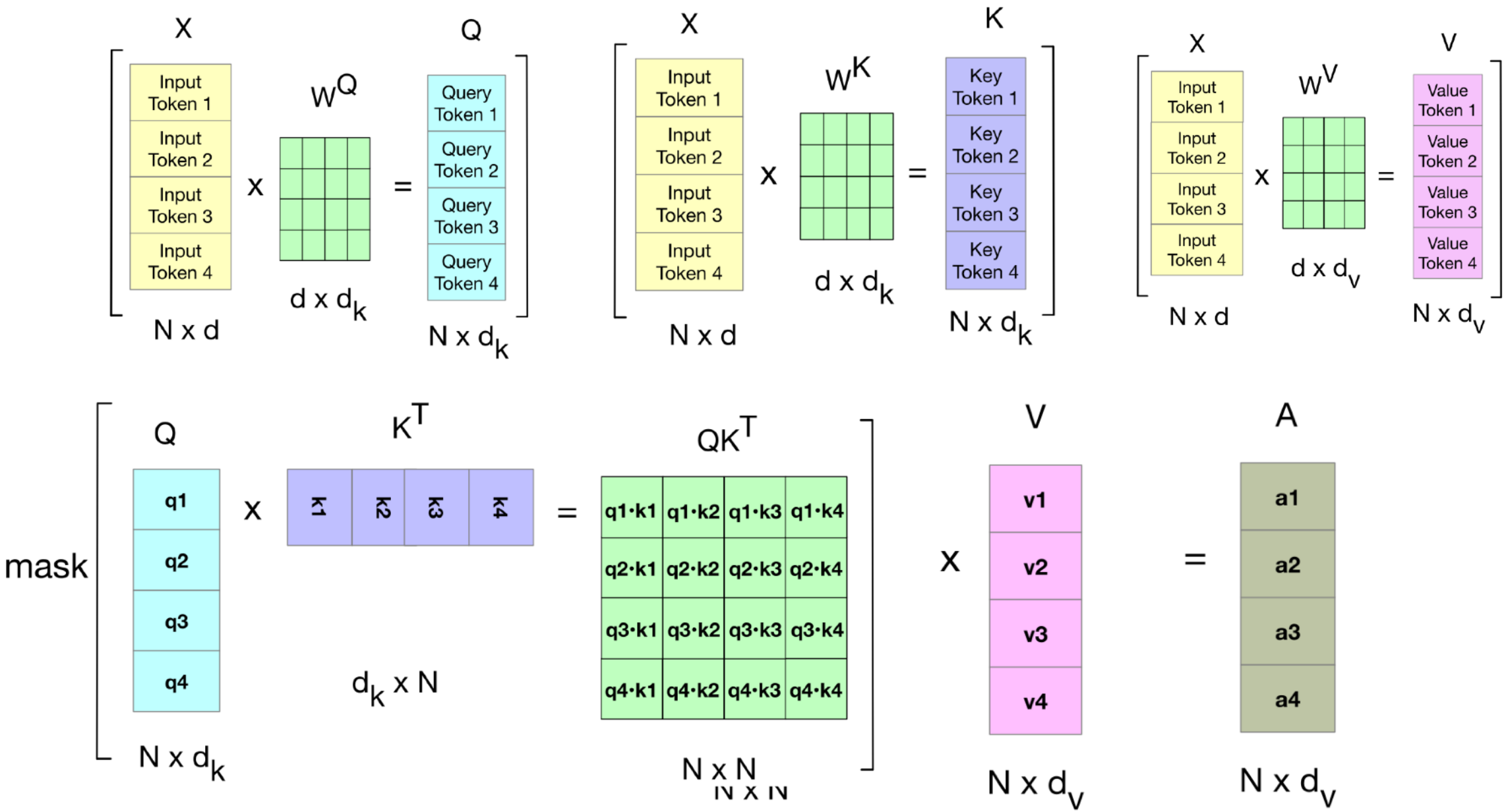
$$\mathbf{A} = \text{softmax}\left(\mathbb{M}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d^k}}\right)\right)\mathbf{V}$$

N

q1·k1	$-\infty$	$-\infty$	$-\infty$
q2·k1	q2·k2	$-\infty$	$-\infty$
q3·k1	q3·k2	q3·k3	$-\infty$
q4·k1	q4·k2	q4·k3	q4·k4

N

Attention encore une fois



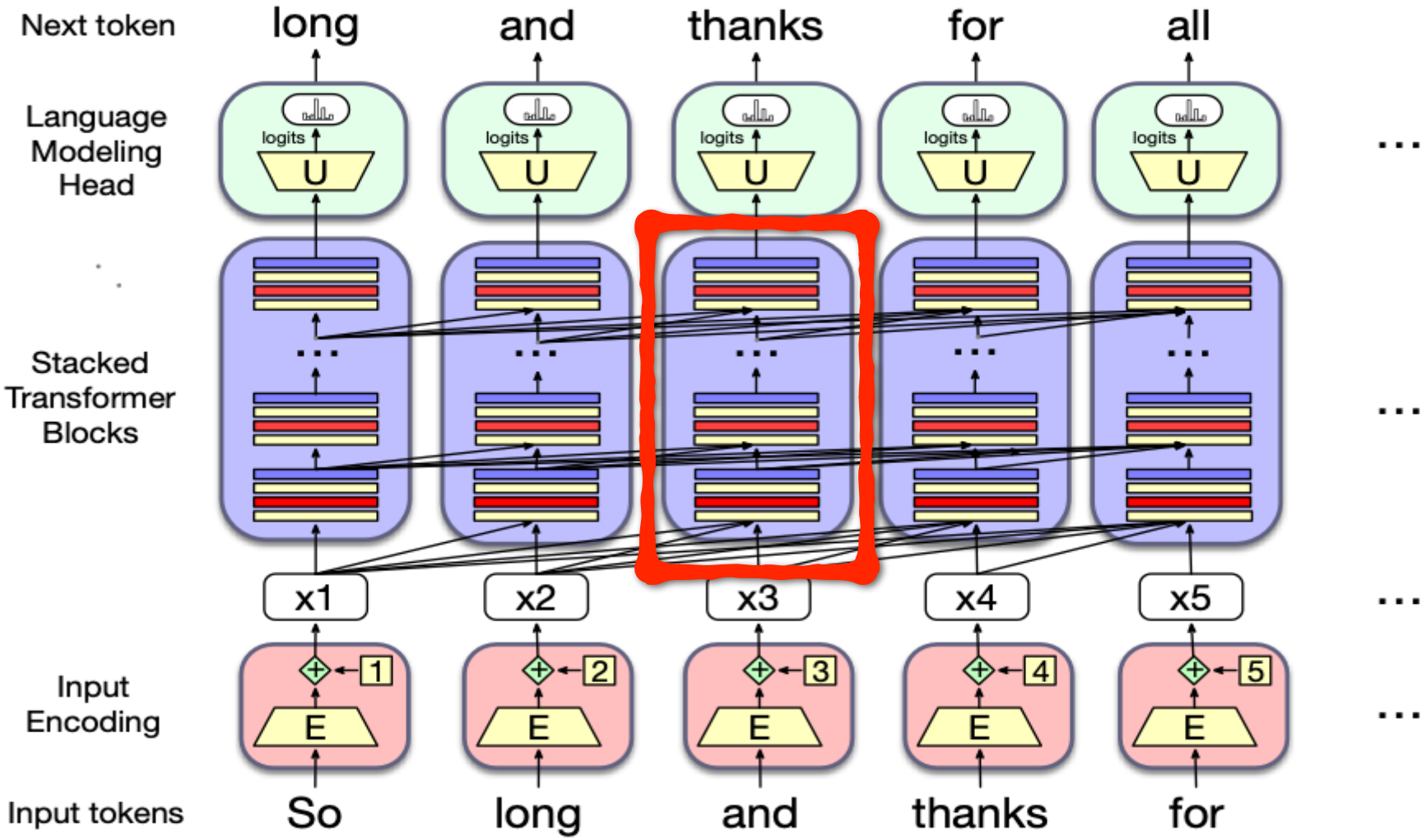
Attention multi-têtes en parallèle

$$\mathbf{Q^i} = \mathbf{XW^{Qi}}\,; \quad \mathbf{K^i} = \mathbf{XW^{Ki}}\,; \quad \mathbf{V^i} = \mathbf{XW^{Vi}}$$

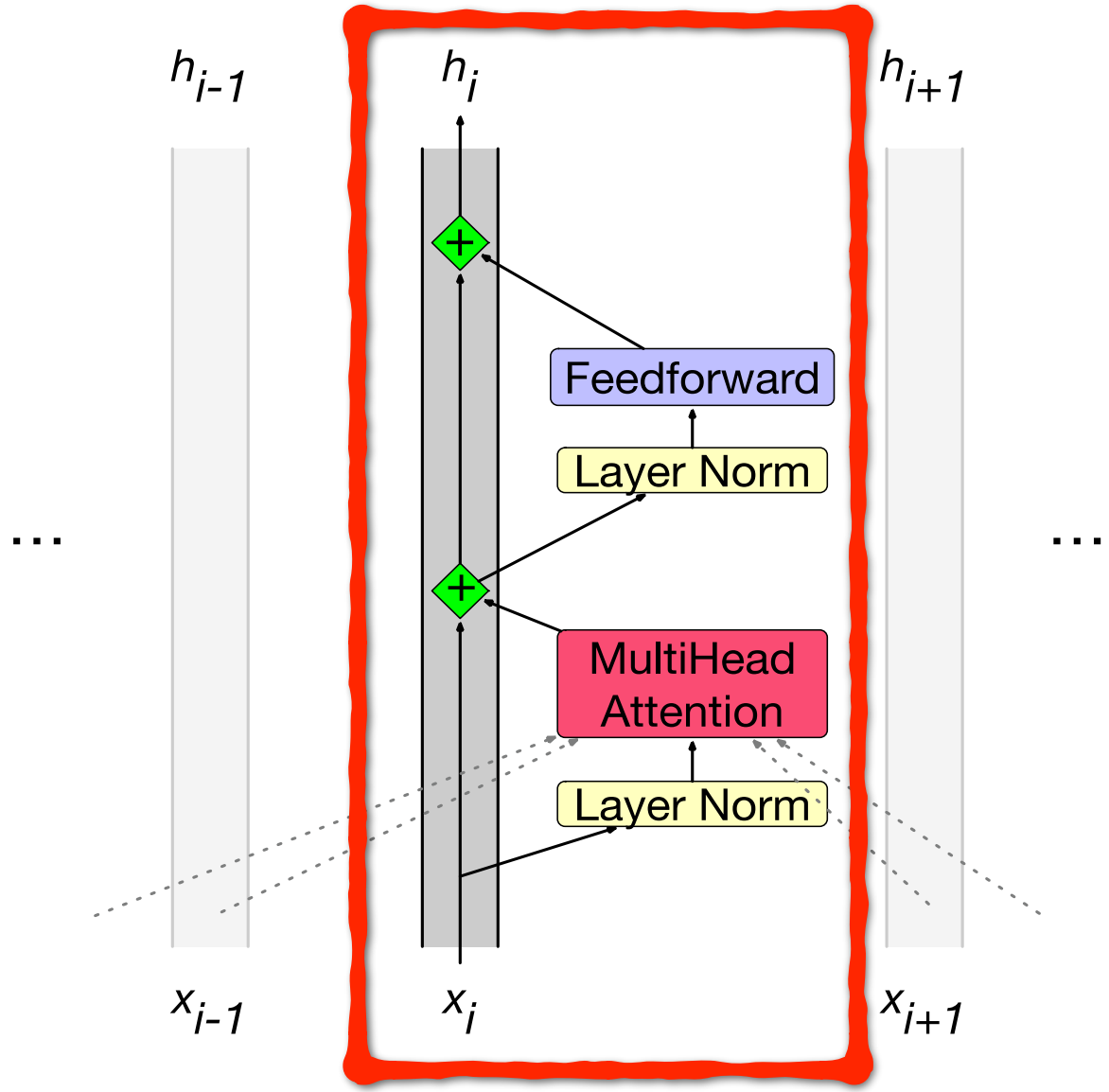
$$\mathbf{head}_i = \mathrm{SelfAttention}(\mathbf{Q^i}, \mathbf{K^i}, \mathbf{V^i}) = \mathrm{softmax}\left(\frac{\mathbf{Q^i}\mathbf{K^i}^{\mathsf{T}}}{\sqrt{d_k}}\right)\mathbf{V^i}$$

$$\mathrm{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \ldots \oplus \mathbf{head}_A)\mathbf{W^O}$$

Rappel : architecture du transformer

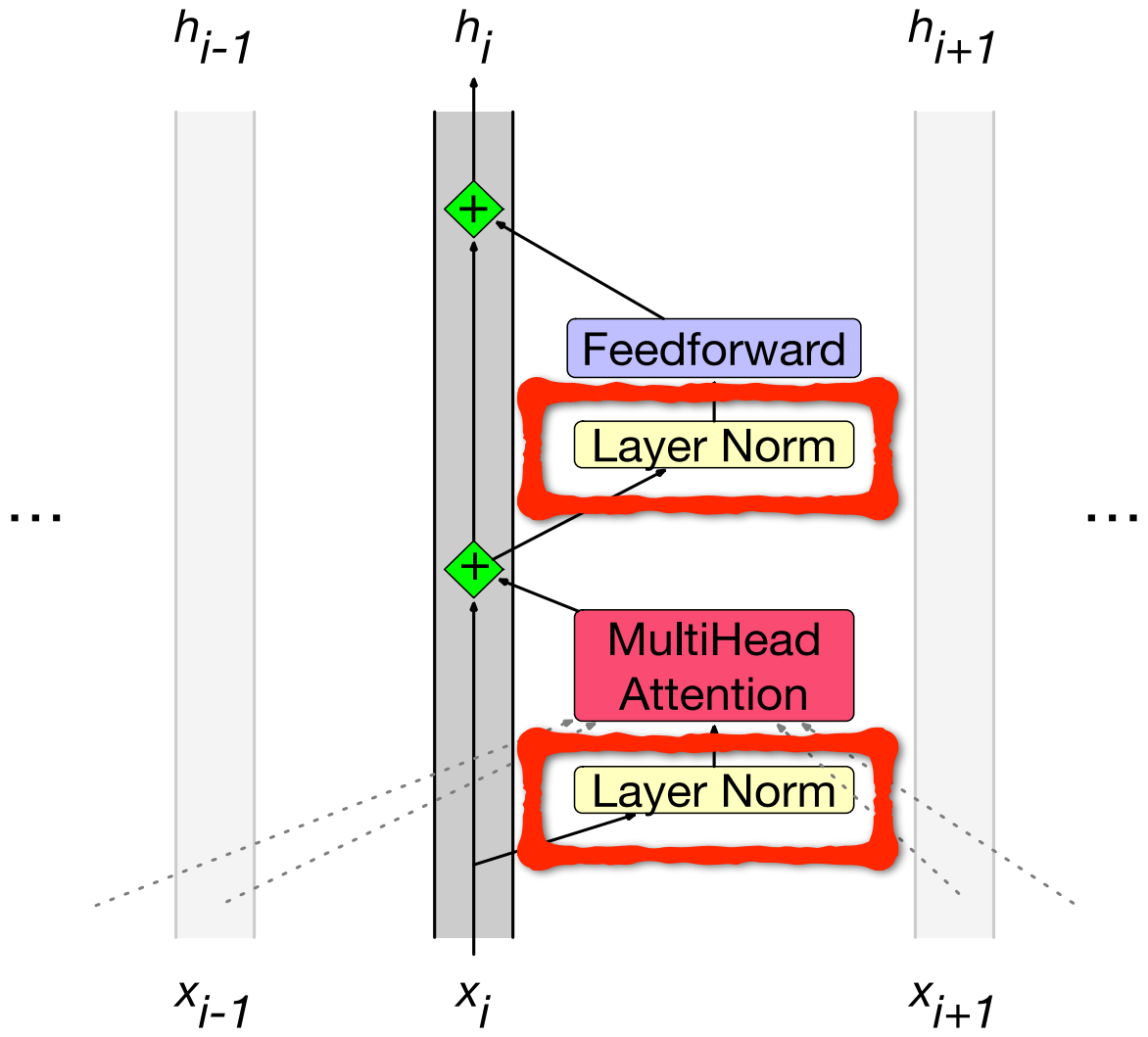


Un bloc transformer



Sous-couches du bloc de transformer : Layer Norm

$$\text{LayerNorm}(\mathbf{x}_i) = \ldots$$



Layer Norm

Layer norm est une variation du score z des statistiques, appliquée à un seul vecteur dans une couche cachée

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

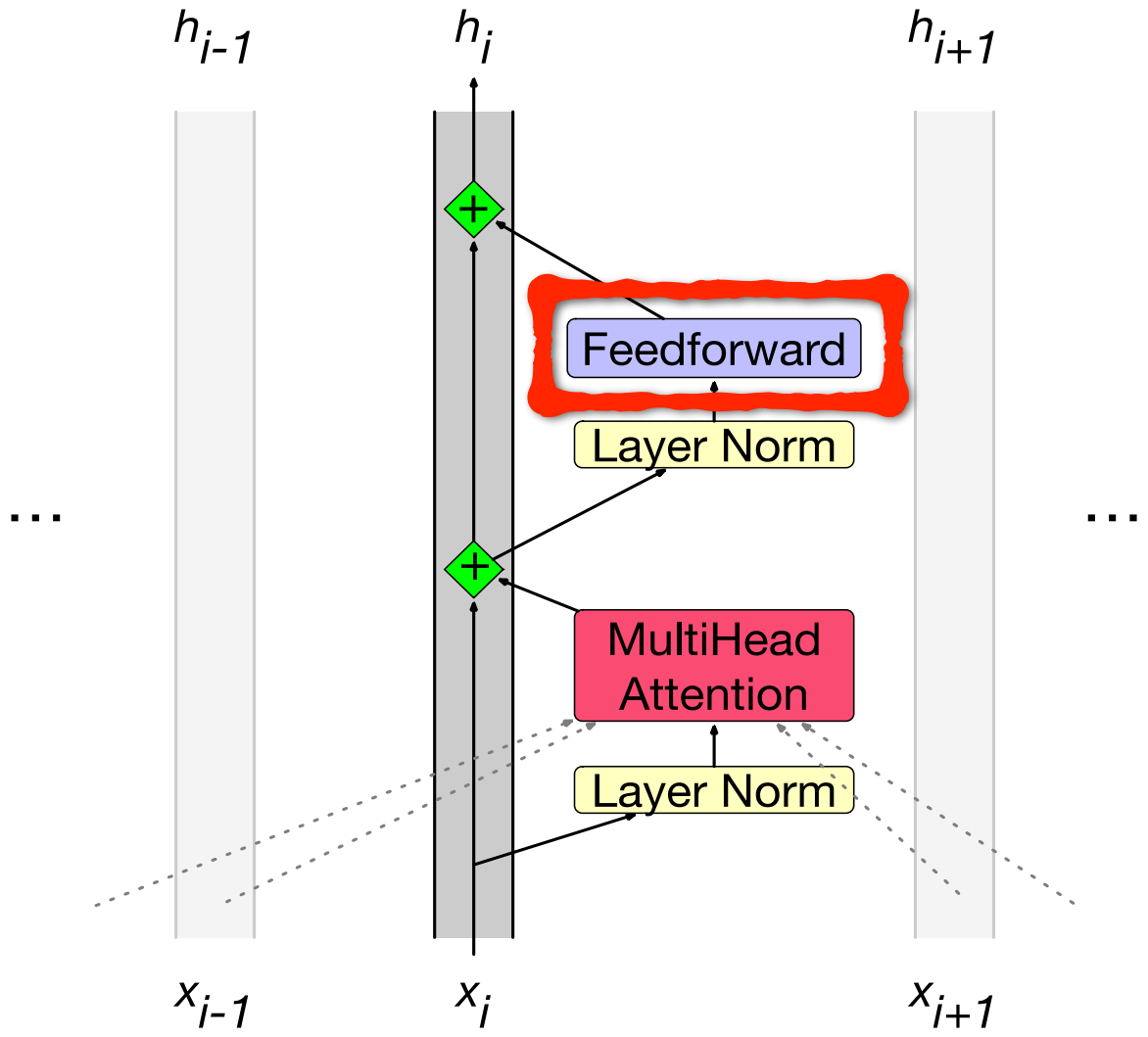
$$\sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

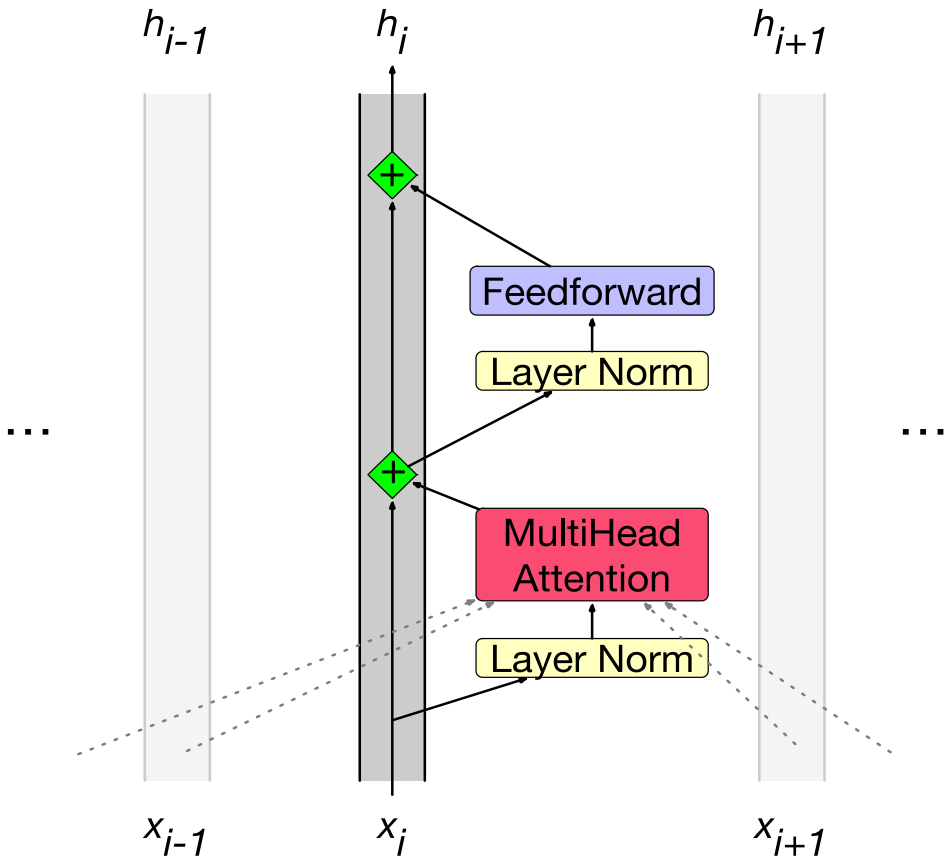
$$\text{LayerNorm}(\mathbf{x}) = \gamma\,\frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$

Sous-couches du bloc de transformer : FFN

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$



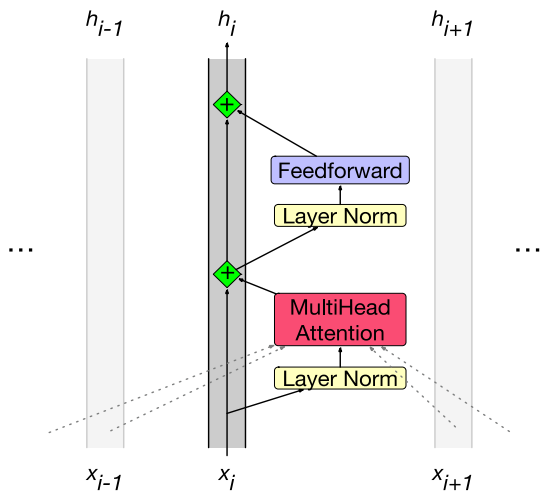
Assembler un seul bloc de transformer



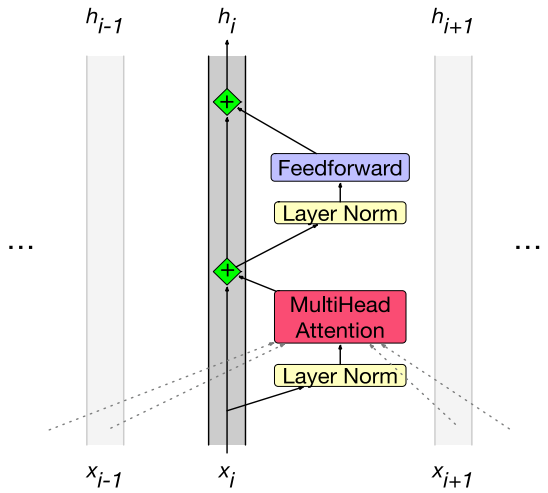
$$
\begin{aligned}
\mathbf{t}_i^1 &= \mathrm{LayerNorm}(\mathbf{x}_i) \\
\mathbf{t}_i^2 &= \mathrm{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{t}_1^1, \cdots, \mathbf{t}_N^1]) \\
\mathbf{t}_i^3 &= \mathbf{t}_i^2 + \mathbf{x}_i \\
\mathbf{t}_i^4 &= \mathrm{LayerNorm}(\mathbf{t}_i^3) \\
\mathbf{t}_i^5 &= \mathrm{FFN}(\mathbf{t}_i^4) \\
\mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3
\end{aligned}
$$

Un transformer est une série de ces blocs, Donc tous les vecteurs sont de la même dimensionnalité *d*

Block 2



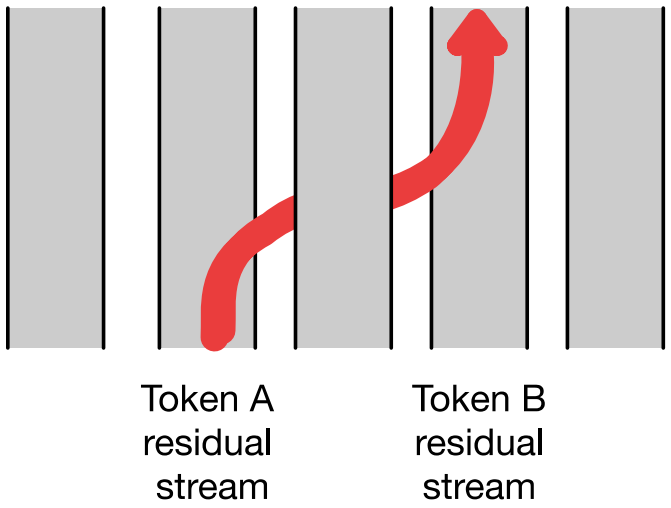
Block 1



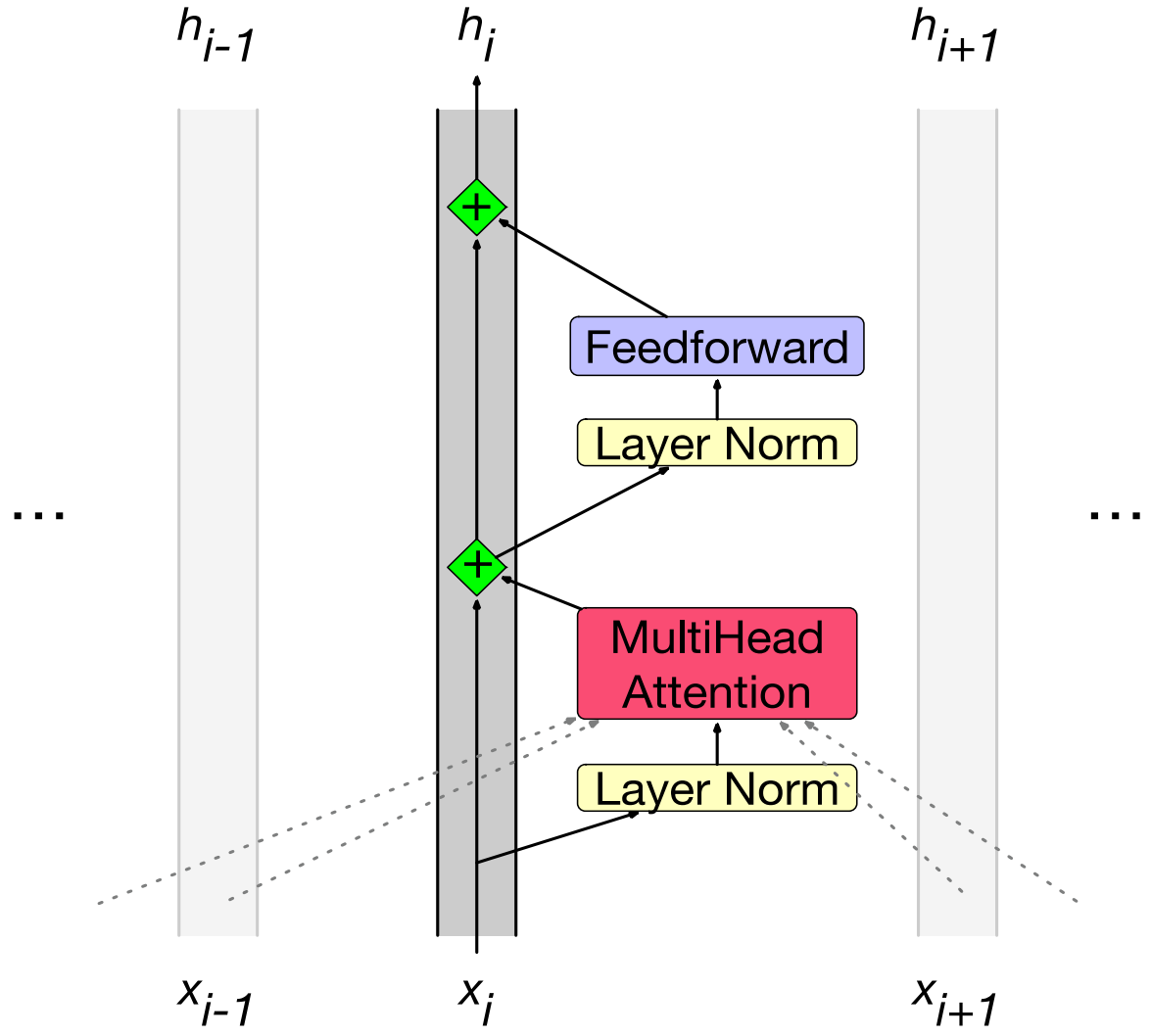
Flux résiduels et attention

Notez que toutes les parties du bloc de transformer s'appliquent à 1 flux résiduel à l'exception de l'attention, qui prend des informations d'autres tokens

Elhage et al. (2021) montre que nous pouvons voir les têtes d'attention comme des informations littéralement déplacées du flux résiduel d'un token voisin dans le flux actuel



Vue du flux résiduel



- FFN et couches d'attention **lisent** et **écrivent** dans le flux résiduel
- Les couches FFN n'ont accès qu'à "une voie". Même calcul appliqué sur chaque « voie »
- Les couches d'attention peuvent également lire à partir d'autres "voies"
- $\mathbf{x}_i$ est transformé en $\mathbf{h}_i^L$ À travers une séquence de transformations non linéaires

Assembler un seul bloc de transformer pour X

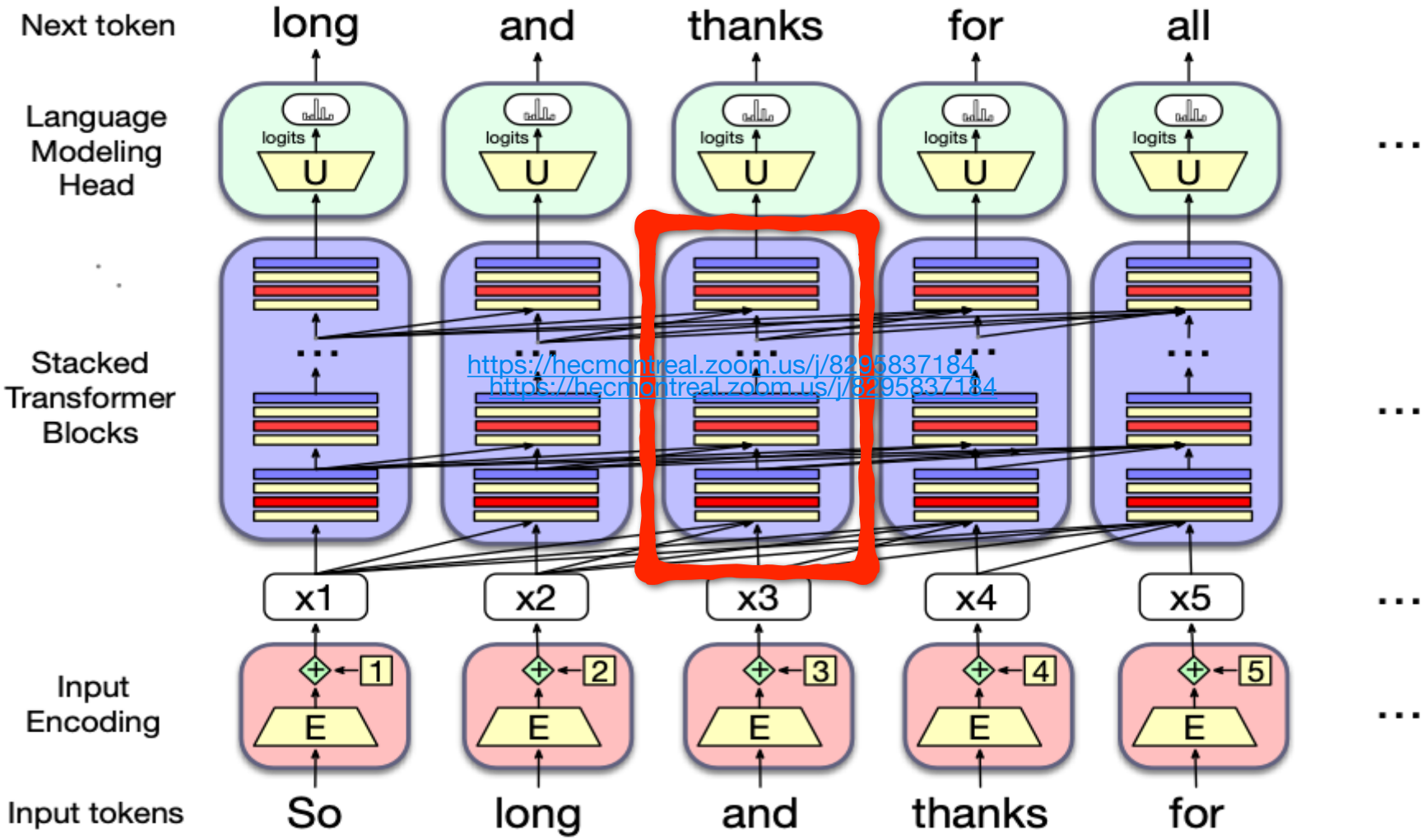
Single vector:

$$
\begin{aligned}
\mathbf{t}_i^1 &= \mathrm{LayerNorm}(\mathbf{x}_i) \\
\mathbf{t}_i^2 &= \mathrm{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{t}_1^1, \cdots, \mathbf{t}_N^1]) \\
\mathbf{t}_i^3 &= \mathbf{t}_i^2 + \mathbf{x}_i \\
\mathbf{t}_i^4 &= \mathrm{LayerNorm}(\mathbf{t}_i^3) \\
\mathbf{t}_i^5 &= \mathrm{FFN}(\mathbf{t}_i^4) \\
\mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3
\end{aligned}
$$

Matrix of inputs:

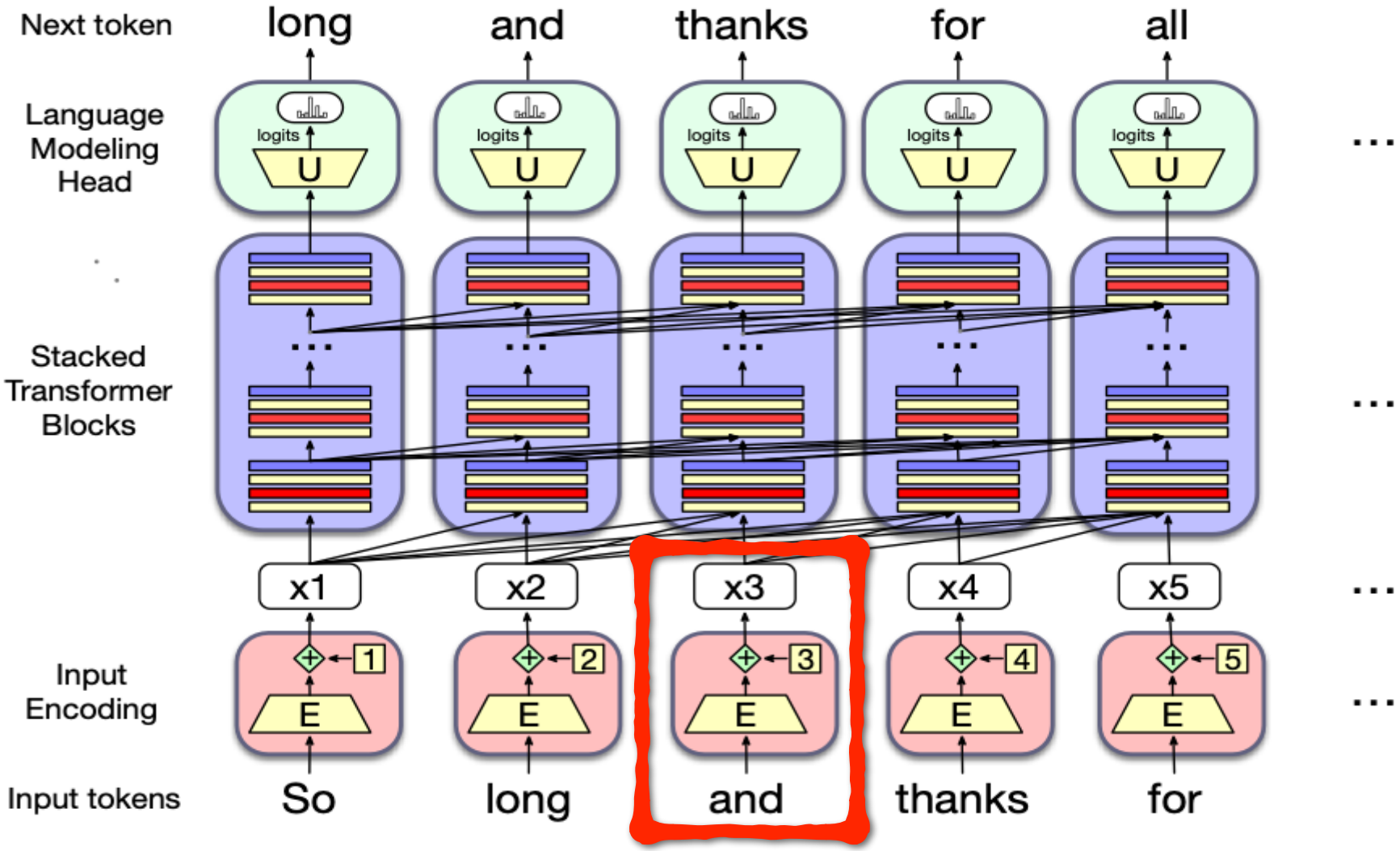
$$
\begin{aligned}
\mathbf{T}^1 &= \mathrm{LayerNorm}(\mathbf{X}) \\
\mathbf{T}^2 &= \mathrm{MultiHeadAttention}(\mathbf{T}^1) \\
\mathbf{T}^3 &= \mathbf{T}^2 + \mathbf{X} \\
\mathbf{T}^4 &= \mathrm{LayerNorm}(\mathbf{T}^3) \\
\mathbf{T}^5 &= \mathrm{FFN}(\mathbf{T}^4) \\
\mathbf{H} &= \mathbf{T}^5 + \mathbf{T}^3
\end{aligned}
$$

Rappel : architecture du transformer



https://hecmontreal.zoom.us/j/8295837184
https://hecmontreal.zoom.us/j/8295837184

Rappel : architecture du transformer

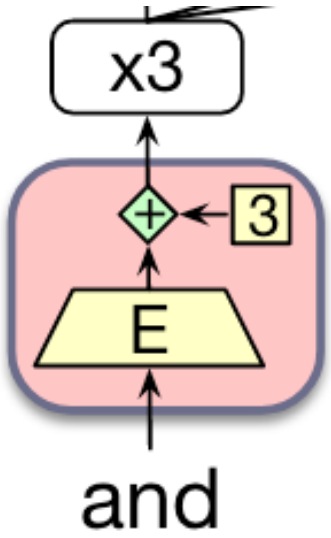


Embeddings du token et positionnel

La matrice $\mathbf{X}$ $(N \times d)$ A un embedding pour chaque mot dans le contexte.

L'embedding Est créé en ajoutant deux embeddings distinctes pour chaque entrée

- embedding du token
- embedding positionnel



Embeddings des tokens

Matrice d'embedding $\mathbf{E}$ de taille $|\mathscr{V}| \times d$.

- Une rangée pour chaqu'un des $|\mathscr{V}|$ tokens du vocabulaire.
- Chaque mot est un vecteur de rangée de dimensions d

Étant donné: le string "*Thanks for all the*"

1. Tokenisez avec BPE et convertir en index du vocabulaire

input_ids = [5,4000,10532,2224]

2. Sélectionnez la rangée correspondante de $\mathbf{E}$, chaque rangée est un embedding

- (rangée 5, rangée 4000, rangée 10532, rangée 2224).

Embeddings positionnels

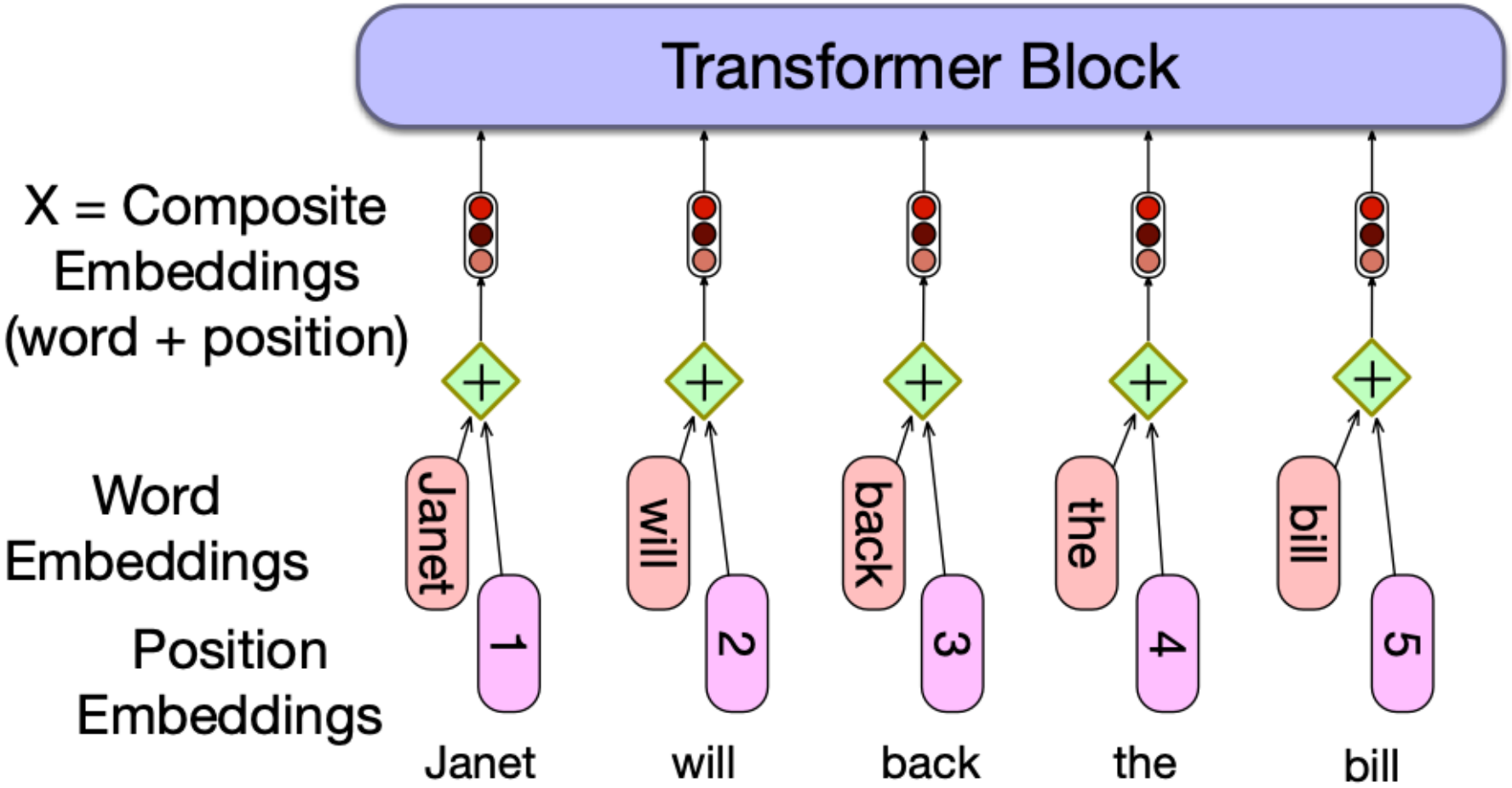
Il existe de nombreuses méthodes, mais nous allons simplement décrire la plus simple : la position absolue.

Objectif : apprendre une matrice d'intégration de position $\mathbf{E}_{\text{pos}}$ de taille $1 \times N$.

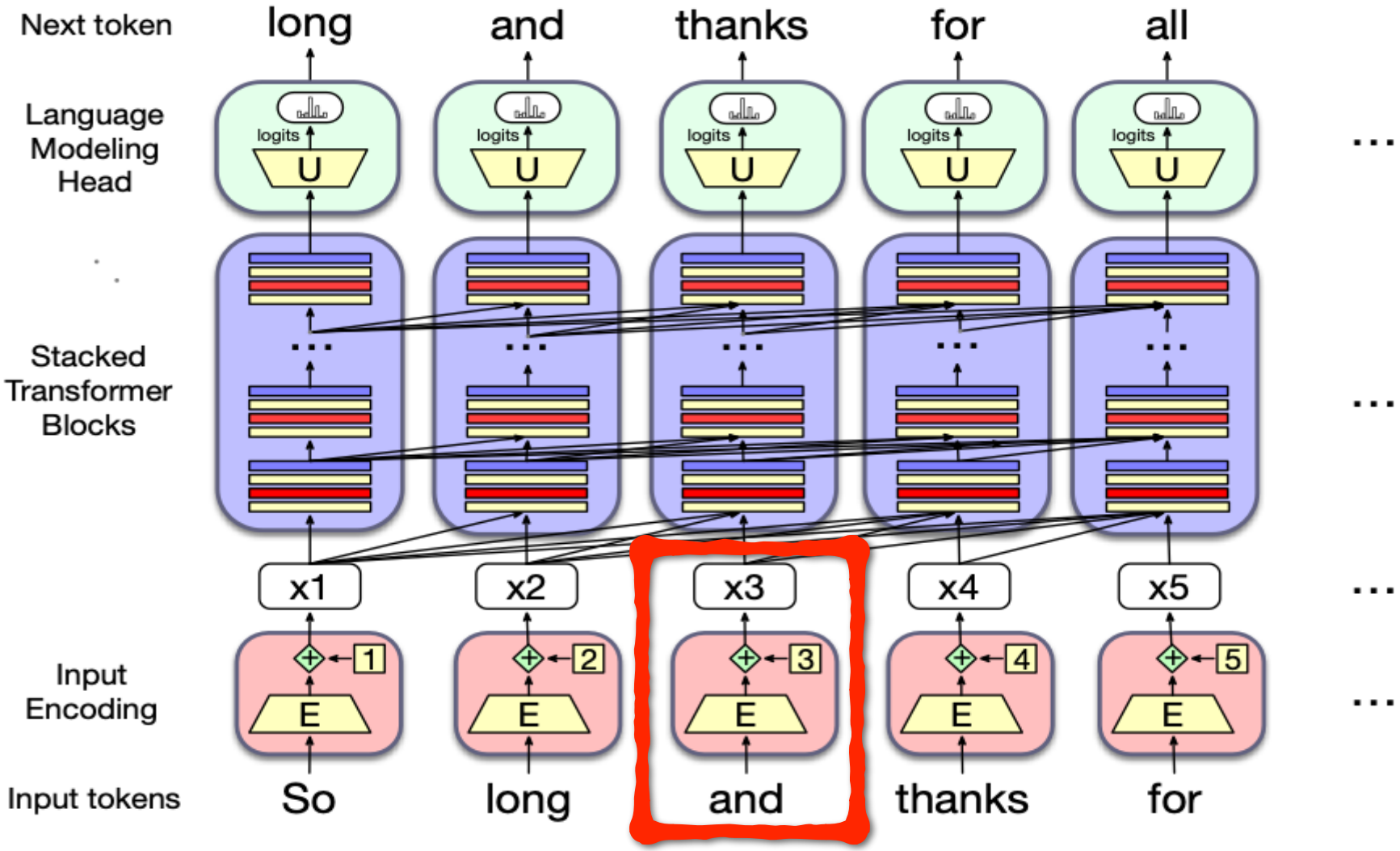
Commencez par des embeddings initialisées aléatoirement

- Un pour chaque nombre entier jusqu'à une certaine longueur maximale.
- i.e., Tout comme nous avons un embedding pour le token *fish*, nous aurons un embedding pour la position 3 et la position 17.
- Comme pour les embeddings des tokens, ces embeddings positionnels sont apprises avec d'autres paramètres pendant l'entraînement.

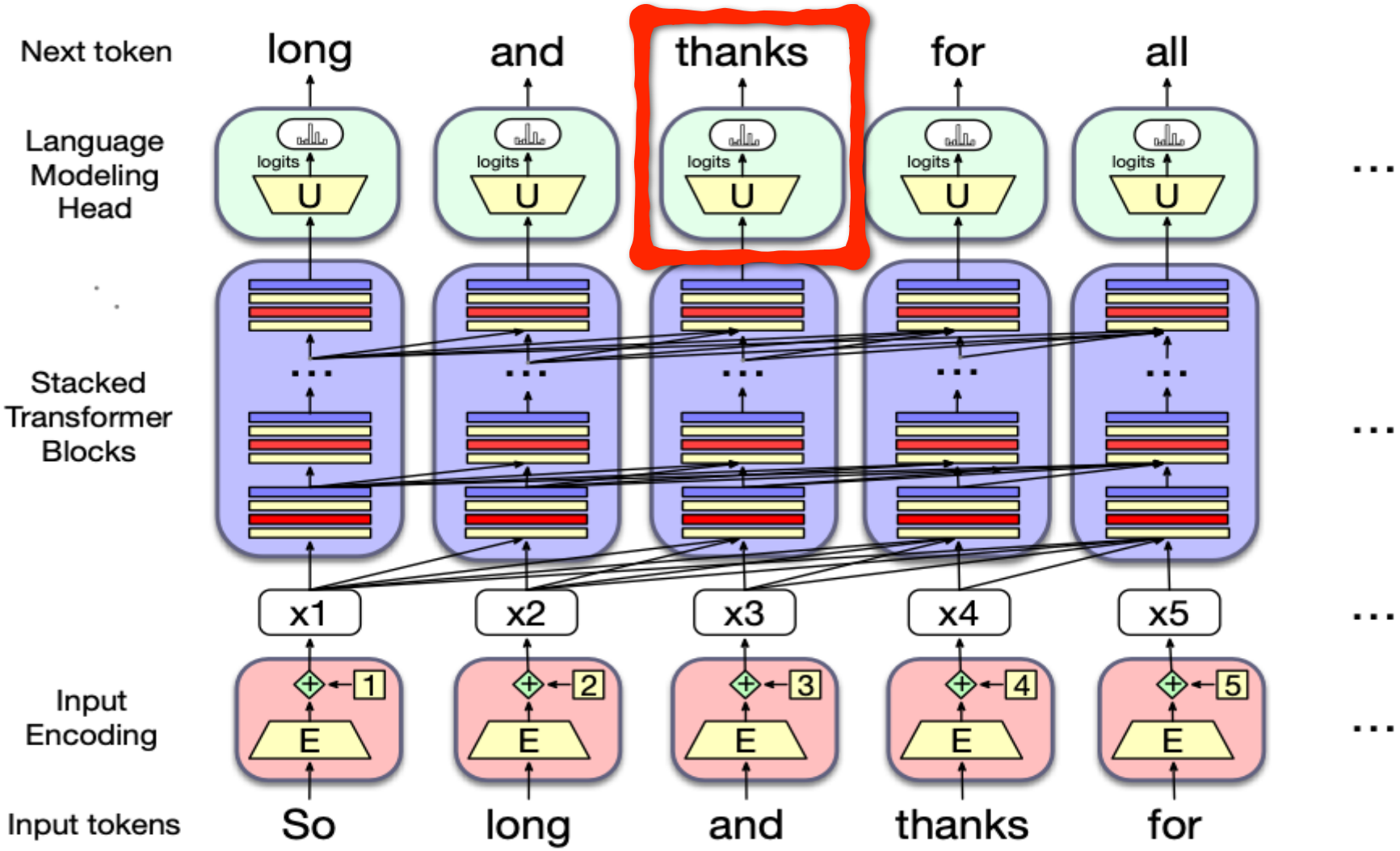
Chaque $\mathbf{x}_i$ est la somme des of embeddings du token et positionnel



Rappel : architecture du transformer



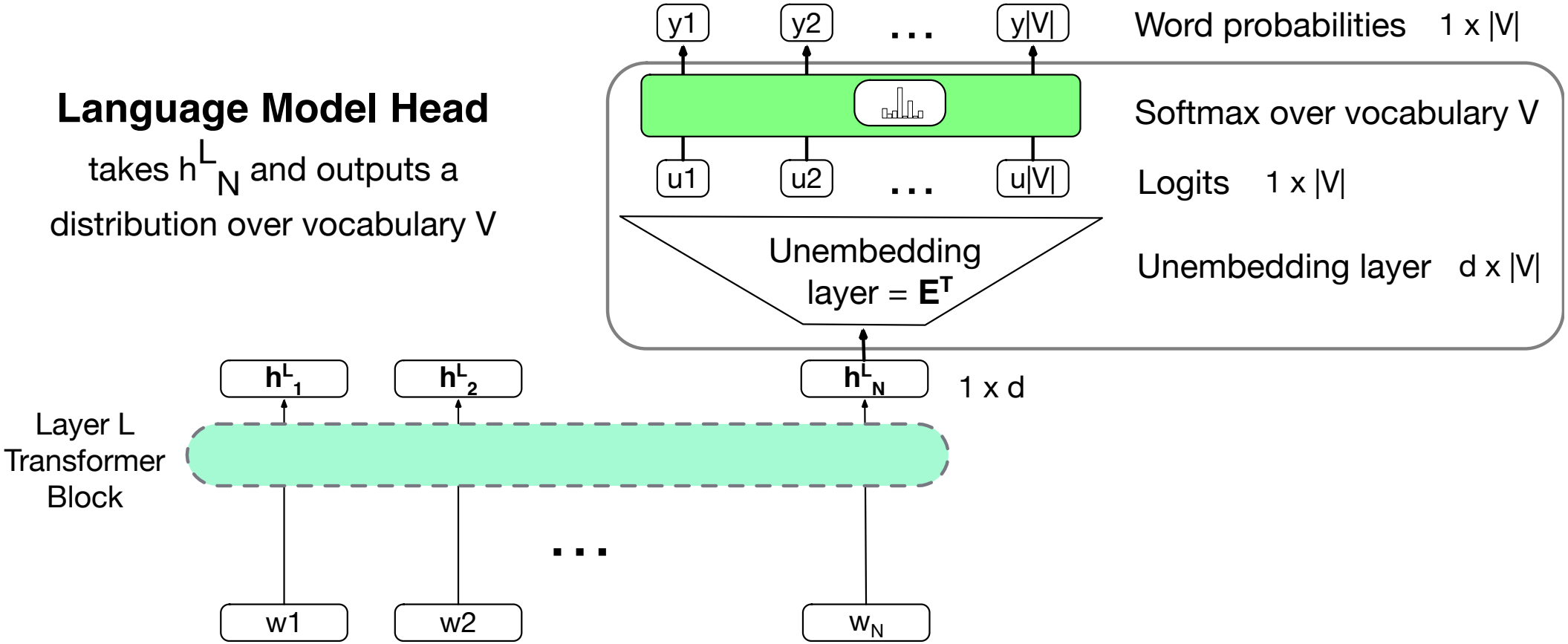
Rappel : architecture du transformer



Tête de modélisation du langage

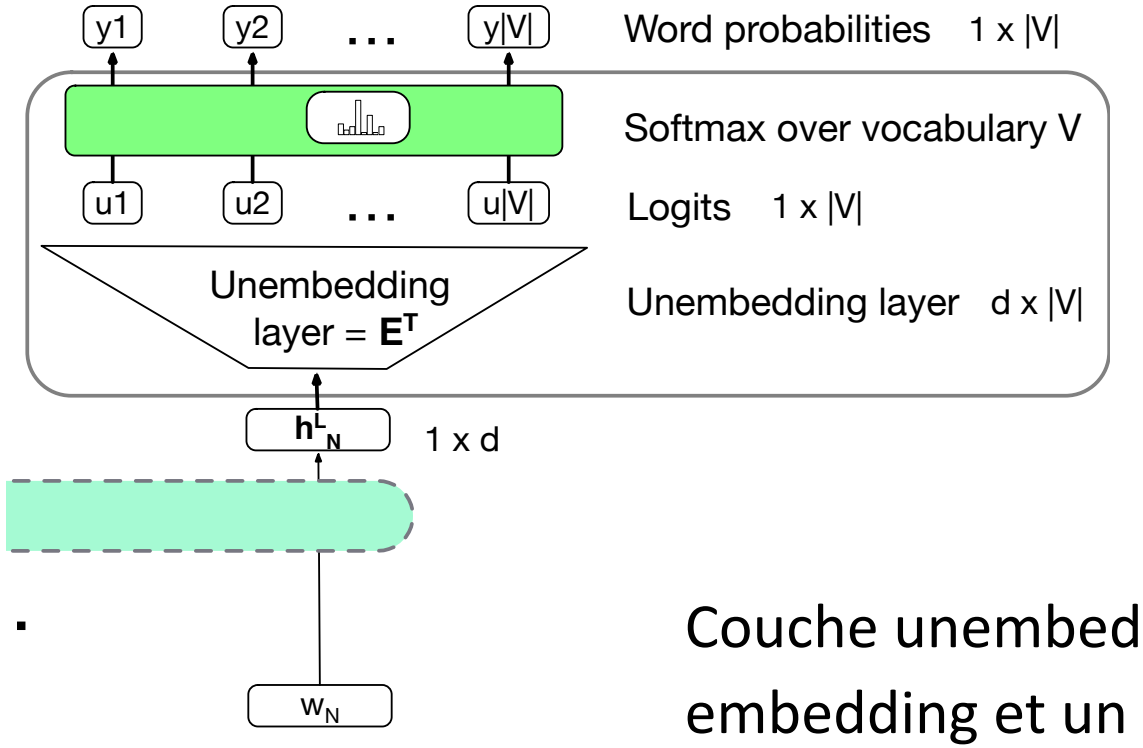
Language Model Head

takes h^L_N and outputs a distribution over vocabulary V



Tête de modélisation du langage

Couche “Unembedding”: couche linéaire qui projette de $\mathbf{h}_N^L$ (shape $1 \times d$) au vecteur de logits



Pourquoi "unembedding"?
Attaché to $\mathbf{E}^T$

Attacher les poids “Weight tying”, Nous utilisons les mêmes poids pour deux matrices différentes

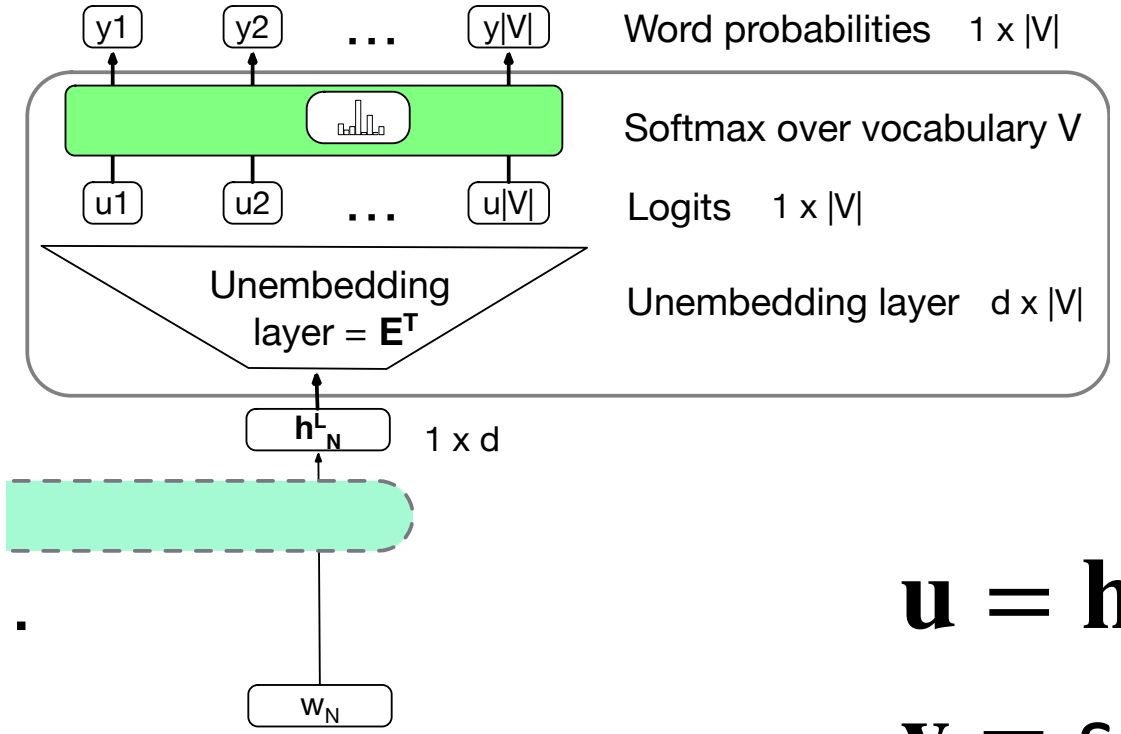
Couche unembedding fait la correspondence entre un embedding et un vecteur de logits $1 \times |\mathscr{V}|$

Tête de modélisation du langage

Logits, le vecteur de score $\mathbf{u}$, de taille $1 \times |\mathscr{V}|$.

Un score pour chacun des $|\mathscr{V}|$ token possible dans le vocabulaire $\mathscr{V}$.

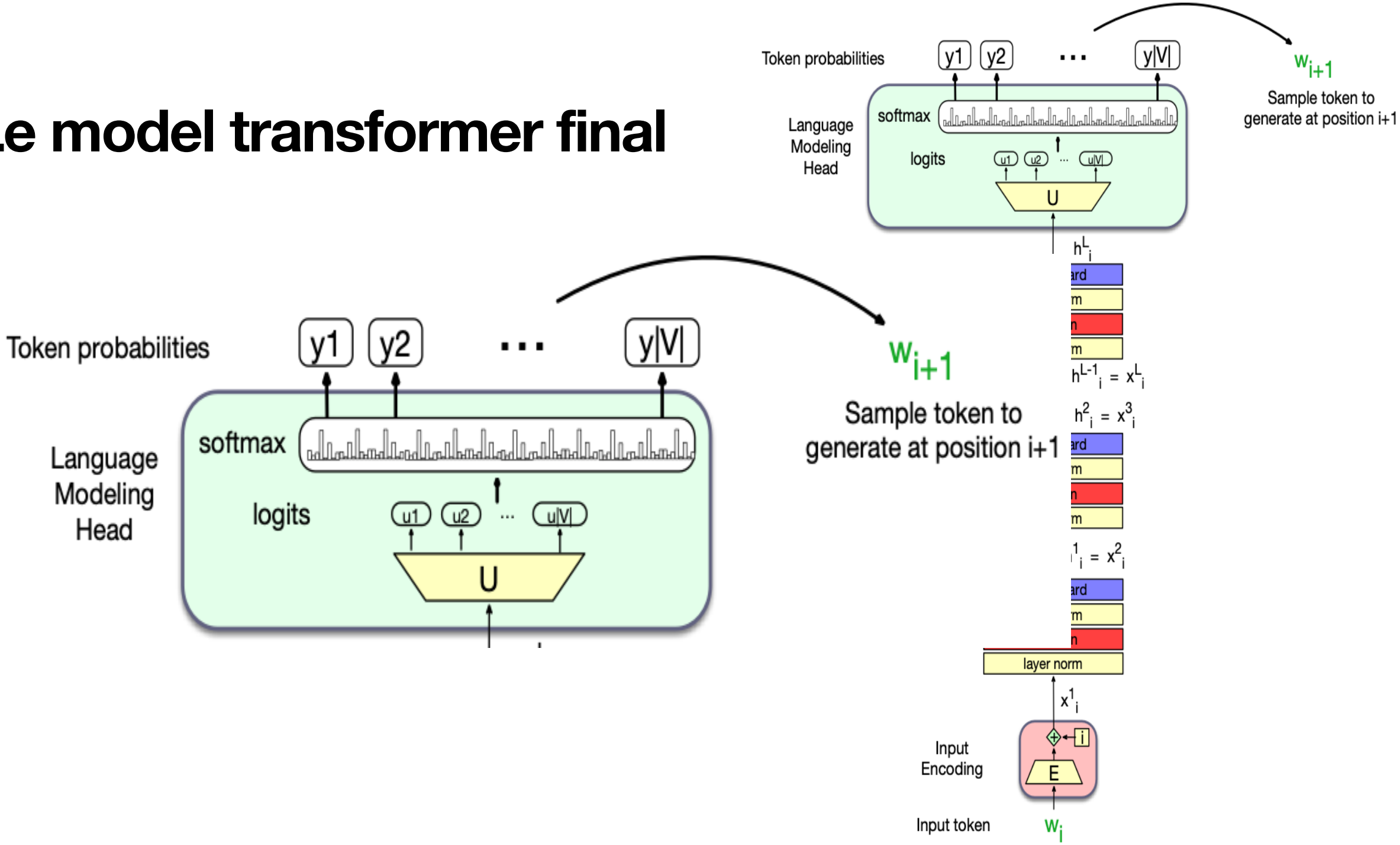
Softmax transforme les logits en probabilités sur le vocabulaire. de taille $1 \times |\mathscr{V}|$.



$$\mathbf{u} = \mathbf{h}_N^L \mathbf{E}^\top$$

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$

Le model transformer final



[Pause - 15 minutes]

Travailler avec des Transformers

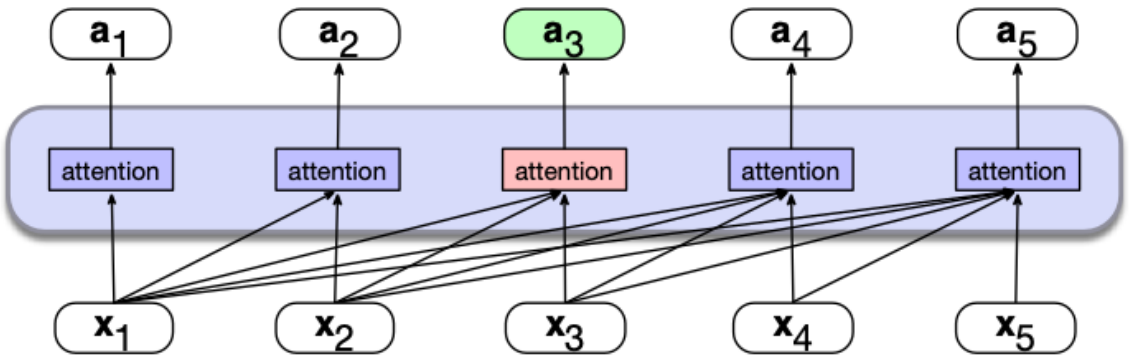
Mettez-vous en équipe!

Ouvrez les exercices/week 7 dans votre dossier de cours et commencez à écrire/exécuter du code !

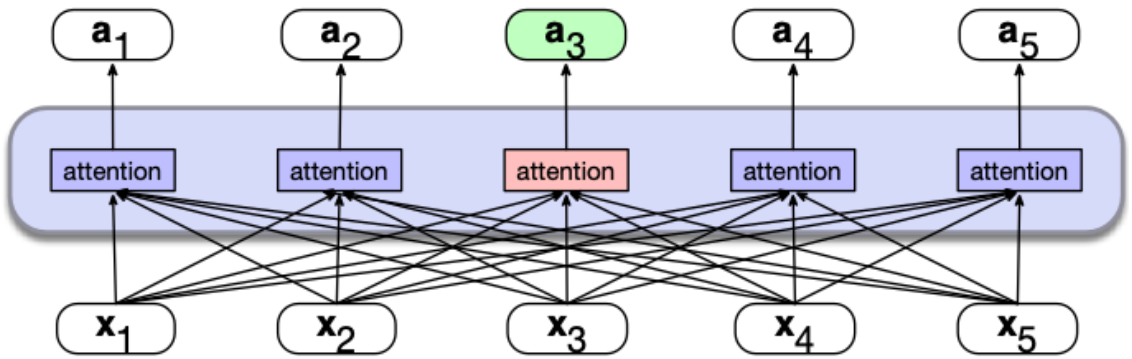
Masked Language Modeling

- We've seen autoregressive (causal, left-to-right) LMs.
- But what about tasks for which we want to peak at future tokens?
  - Especially true for tasks where we map each input token to an output token
- **Bidirectional encoders** use **unmasked self-attention** to
  - map sequences of input embeddings $\mathbf{x}_1, \ldots, \mathbf{x}_n$
  - to sequences of output embeddings of the same length $\mathbf{h}_1, \ldots, \mathbf{h}_n$
  - where the output vectors have been contextualized using information from the entire input sequence.

Bidirectional Self-Attention



a) A causal self-attention layer



b) A bidirectional self-attention layer

We just remove the mask

Casual self-attention

N

q1·k1	−∞	−∞	−∞
q2·k1	q2·k2	−∞	−∞
q3·k1	q3·k2	q3·k3	−∞
q4·k1	q4·k2	q4·k3	q4·k4

N

$$\mathbf{A} = \text{softmax}\left(\mathbb{M}\left(\frac{\mathbf{QK}^\top}{\sqrt{d^k}}\right)\right)$$

Bidirectional self-attention

N

q1·k1	q1·k2	q1·k3	q1·k4
q2·k1	q2·k2	q2·k3	q2·k4
q3·k1	q3·k2	q3·k3	q3·k4
q4·k1	q4·k2	q4·k3	q4·k4

N

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d^k}}\right)$$

Bidirectional Encoder Representations from Transformers

BERT (Devlin et al., 2019)

- 30,000 English-only tokens (WordPiece tokenizer)
- Input context window N=512 tokens, and model dimensionality d=768
- L=12 layers of transformer blocks, each with A=12 (bidirectional) multihead-attention layers.
- The resulting model has about 100M parameters.

XLM-RoBERTa (Conneau et al., 2020)

- 250,000 multilingual tokens (SentencePiece Unigram LM tokenizer)
- Input context window N=512 tokens,model dimensionality d=1024
- L=24 layers of transformer blocks, with A=16 multihead attention layers each
- The resulting model has about 550M parameters.

Masked training intuition

- For **left-to-right (causal; decoder-only) LMs**, the model tries to predict the last word from prior words:

  The water of Walden Pond is so beautifully _____

  - And we train it to improve its predictions.
- For **bidirectional masked LMs**, the model tries to predict one or more missing words from all the rest of the words:

  The _____ of Walden Pond _____ so beautifully blue

  - The model generates a probability distribution over the vocabulary for each missing token
  - We use the cross-entropy loss from each of the model's predictions to drive the learning process.

MLM training in BERT

15% of the tokens are randomly chosen to be part of the masking

Example: "Lunch was **delicious**", if delicious was randomly chosen:

Three possibilities:

1. 80%: Token is replaced with special token [MASK]

   Lunch was **delicious** -> Lunch was **[MASK]**

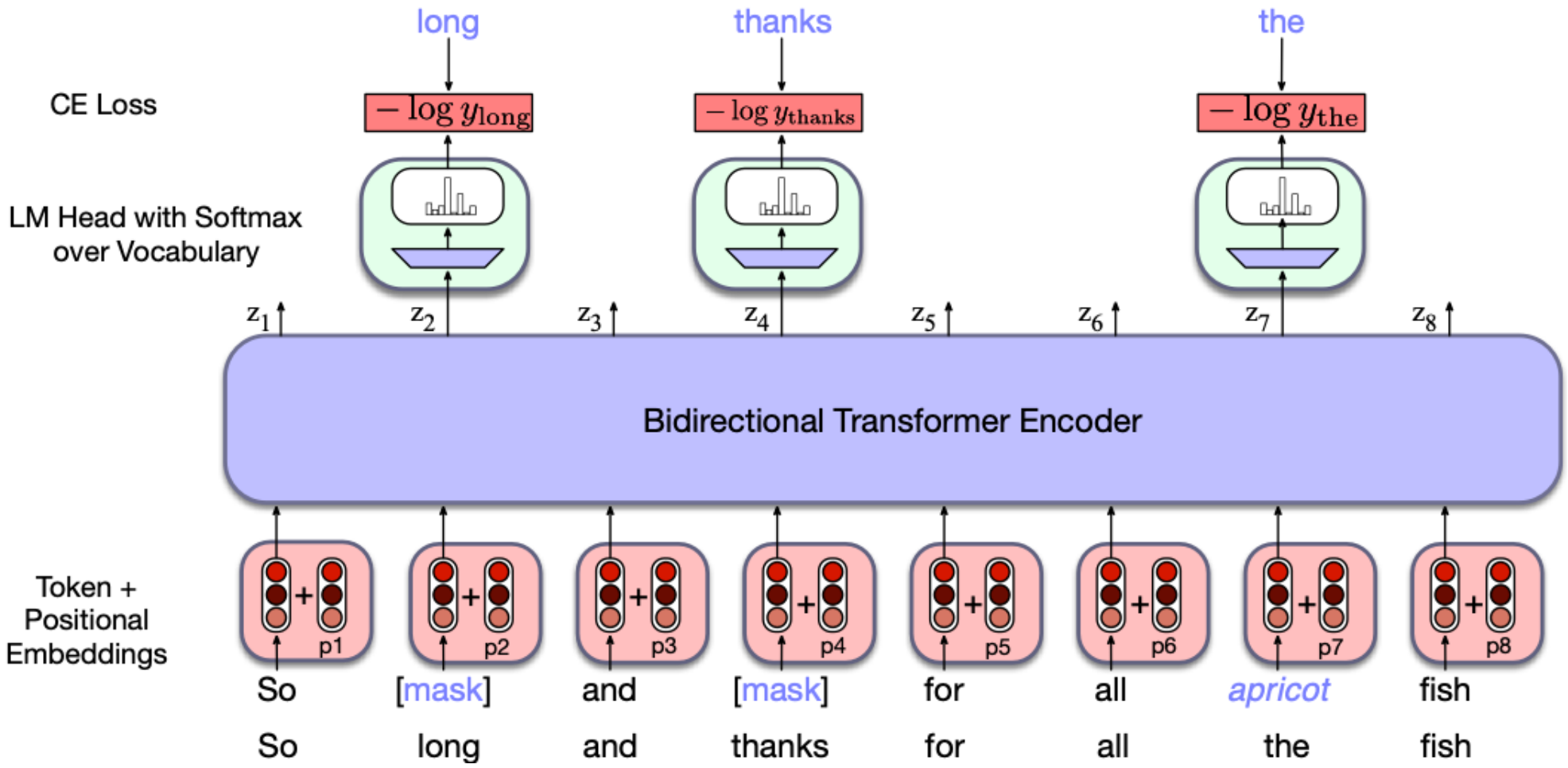
2. 10%: Token is replaced with a random token (sampled from unigram prob)

   Lunch was **delicious** -> Lunch was **gasp**

3. 10%: Token is unchanged

   Lunch was **delicious** -> Lunch was **delicious**

In detail


long
thanks
the
CE Loss
$-\log y_{\text{long}}$
$-\log y_{\text{thanks}}$
$-\log y_{\text{the}}$
LM Head with Softmax over Vocabulary
z_1 z_2 z_3 z_4 z_5 z_6 z_7 z_8
Bidirectional Transformer Encoder
Token + Positional Embeddings
p1 p2 p3 p4 p5 p6 p7 p8
So [mask] and [mask] for all apricot fish
So long and thanks for all the fish

MLM loss

The LM head takes output of final transformer layer L, multiplies it by unembedding layer and turns into probabilities:

$$\mathbf{u}_i = E\mathbf{h}_i^L \qquad\qquad \mathbf{y}_i = \text{softmax}(\mathbf{u}_i)$$

E.g., for the x_i corresponding to "long", the loss is the probability of the correct word *long*, given output h^L_i):

$$\mathscr{L}_{\text{MLM}}(x_i) = -\log P(x_i \mid \mathbf{h}_i^L)$$

We get the gradients by taking the average of this loss over the batch

TODO: there is one more sum: over the different masked positions in the sequence

$$\mathscr{L}_{\text{MLM}}(x_i) = -\frac{1}{|\mathscr{B}|}\sum_{i\in\mathscr{B}} \log P(x_i \mid \mathbf{h}_i^L)$$

Next Sentence Prediction

Given 2 sentences the model predicts if they are a real pair of adjacent sentences from the training corpus or a pair of unrelated sentences.

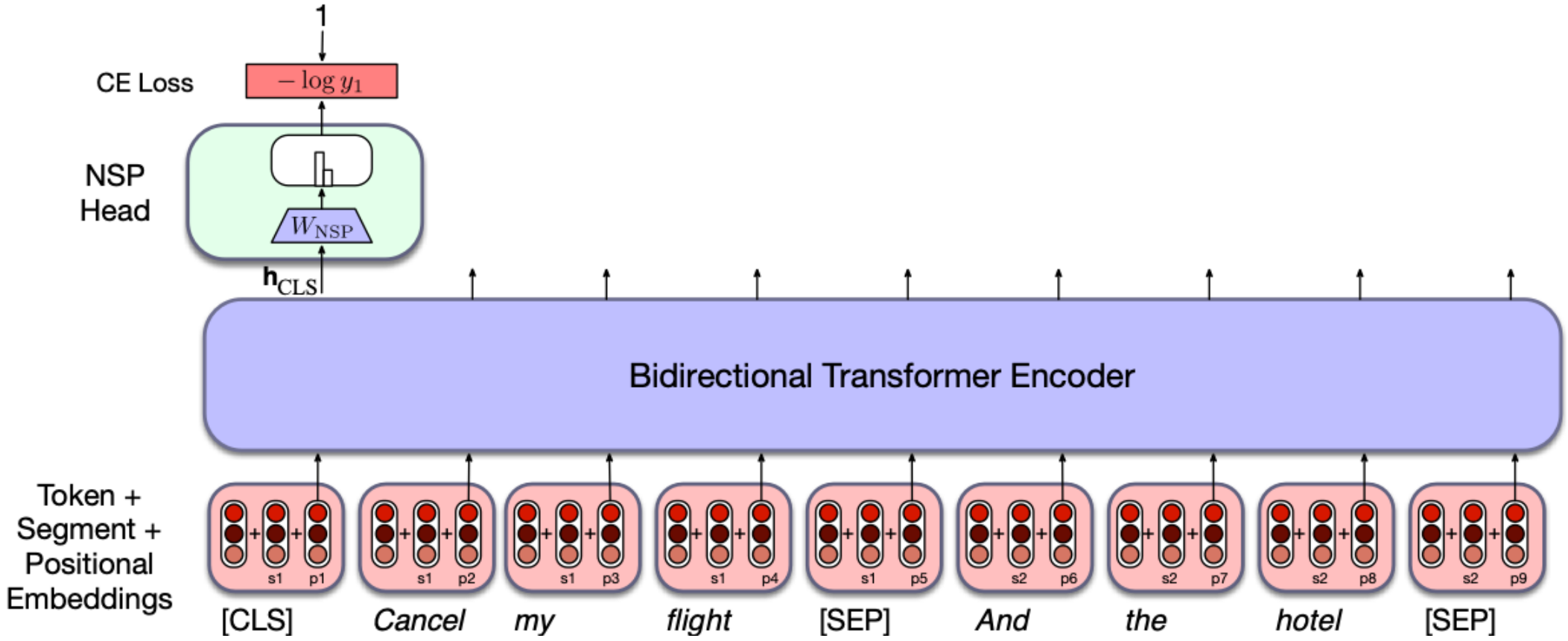
BERT introduces two special tokens

- [CLS] is prepended to the input sentence pair,
- [SEP] is placed between the sentences, and also after second sentence

And two more special tokens

- [1st segment] and [2nd segment]
- These are added to the input embedding and positional embedding

h^L_{CLS} from the final layer [CLS] token is input to classifier head (weights W_{NSP}) that predicts two classes:.

$$\mathbf{y}_i = \text{softmax}(\mathbf{h}^L_{\text{CLS}})\mathbf{W}_{\text{NSP}}$$

NSP Loss with classification head